Chapter 5: Neural Networks – Fundamentals and Applications in Chemical Modeling

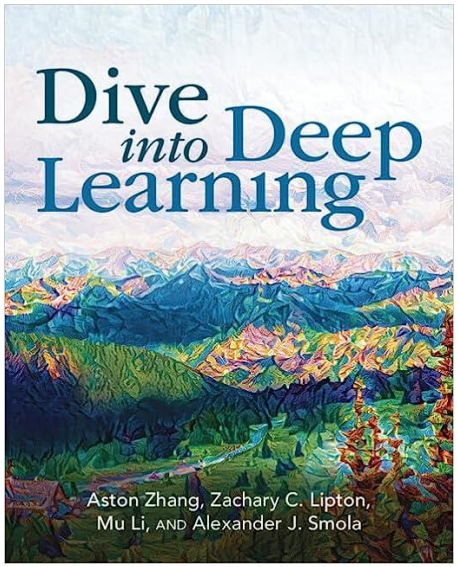
Chem 361

Xuhui Huang

Outline

- **Linear Models as Neural Networks**
- Multilayer Perceptron (MLP): A Nonlinear Classifier
  - Activation Functions
  - Forward Propagation
  - Backward Propagation
- Gradient Vanishing or Exploding
- Techniques to Prevent Overfitting

Reading material:

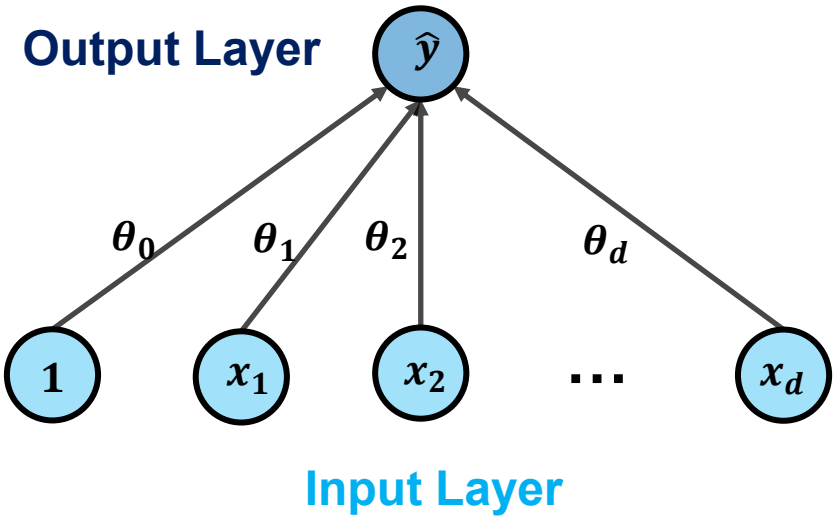


Chapter 3-4

https://d2l.ai/

Linear Regression as a Neural Network

Single-layer neural network:



Linear regression model:

$$\hat{y} = f_\theta(x) = \sum_{i=0}^{d} \theta_i x_i = \theta^T x$$

$$= \theta_0 \cdot 1 + \theta_1 x_1 + \theta_2 x_2 + \cdots \theta_d x_d$$

$\boldsymbol{d}$: feature dimensionality

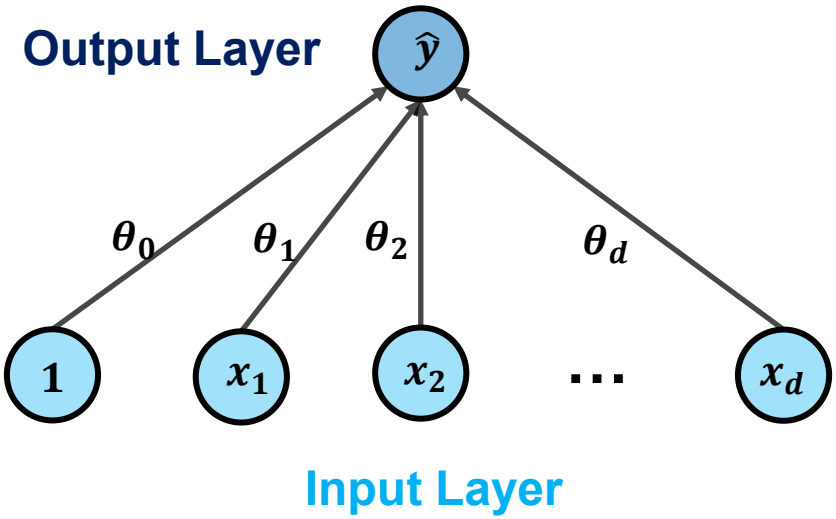
$\boldsymbol{\theta}$: weights

Linear regression = A single-layer fully connected neural network.

A single computed neuron processes the given input values ($x_1, \ldots x_d$) and produces one numerical output ($\hat{y}$ or o_1)!

Linear Regression as a Neural Network

Single-layer neural network:



Linear regression model:

$$\hat{y} = f_\theta(x) = \sum_{i=0}^{d} \theta_i x_i = \theta^T x$$

$$= \theta_0 \cdot 1 + \theta_1 x_1 + \theta_2 x_2 + \cdots \theta_d x_d$$

$\boldsymbol{d}$: feature dimensionality

$\boldsymbol{\theta}$: weights

Linear regression = A single-layer fully connected neural network.

Example: Diazinon

- **Input values** of 4 features (MW, Hdonor, HAcceptor, Rings):

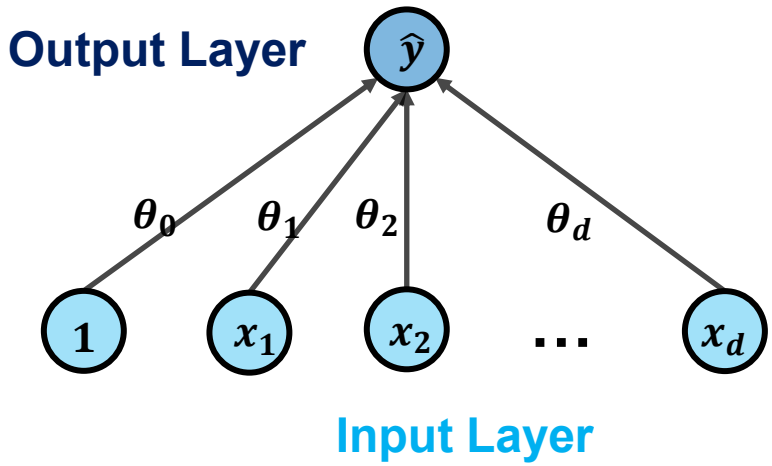
$$(x_1, \ldots x_4) = (304.35,\ \ 0,\ \ 6,\ \ 1)$$

- **Find optimal weights:** θ

- **Output:** predicted solubility (LogS): $\hat{y} = o_1 =$ -3.60

Linear Regression as a Neural Network

Single-layer neural network:



Linear regression model:

$$\hat{y} = f_\theta(x) = \sum_{i=0}^{d} \theta_i x_i = \theta^T x$$

$$= \theta_0 \cdot 1 + \theta_1 x_1 + \theta_2 x_2 + \cdots \theta_d x_d$$

Loss function:

- **Minimize** mean squared error (MSE):

$$\ell(f_\theta) = \frac{1}{n} \sum_{j=1}^{n} \left(f_\theta\left(\boldsymbol{x}^{(j)}\right) - y^{(j)} \right)^2$$

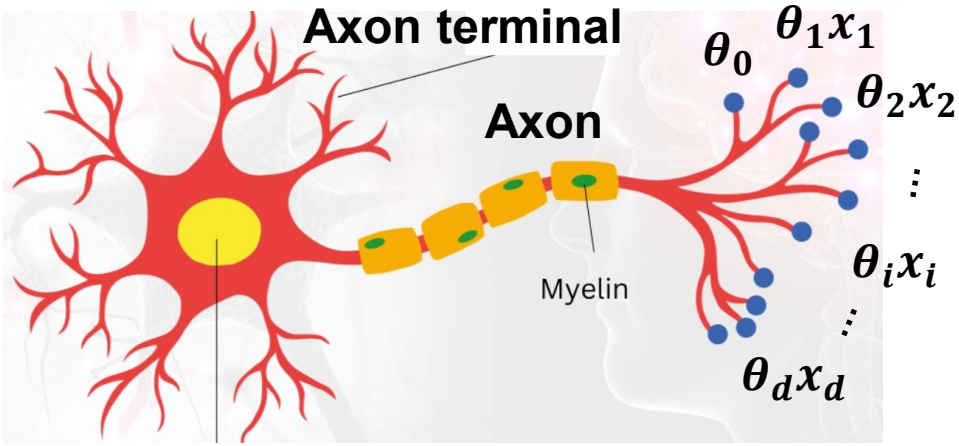
- **Minimize** negative log likelihood – MLE approach:

$$\ell_{MLE} = -\sum_{j=1}^{n} \log p\left(y^{(j)} | x^{(j)}, \theta\right)$$

The two loss functions are equivalent for linear regression when the noise follows a Gaussian distribution. – *MLE is commonly used to derive loss functions in ML.*

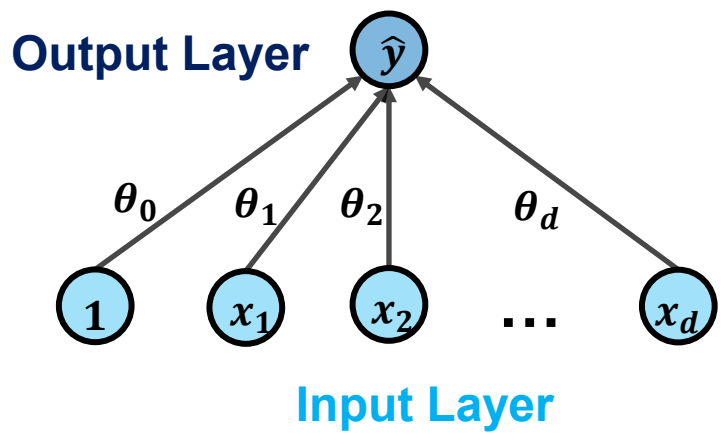
Linear Regression as a Neural Network

A neuron:



https://formalpsychology.com/

Single-layer neural network:

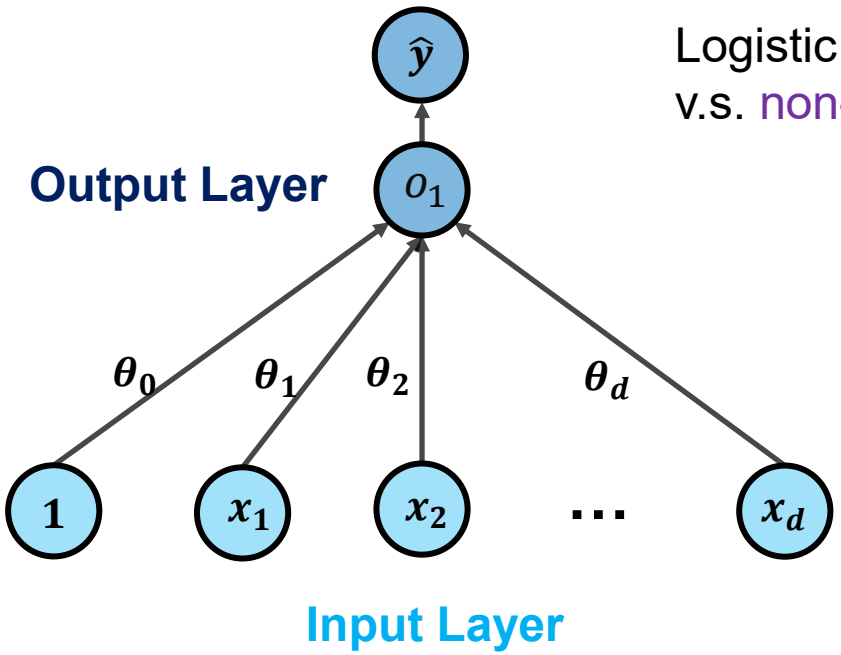


Dendrites receive information x_i from another neuron with a synaptic weight θ_i. – Total input: weighted sum from d resources: $\hat{y} = \theta_0 \cdot 1 + \theta_1 x_1 + \cdots \theta_d x_d$

Axon post-processes its input by applying an activation function $\sigma(\hat{y})$, often non-linear. In a linear regression neural network, $\sigma(\hat{y}) = \hat{y}$ (identity function).

Axon terminal transmits the output to its destination, such as other neurons.

Linear Neural Networks for Classification



Logistic regression for binary prediction (e.g., soluble v.s. non-soluble):

$$\hat{y} = \text{Sigmoid}\left(\sum_{i=0}^{d} \theta_i x_i\right) = \text{Sigmoid}(o_1)$$

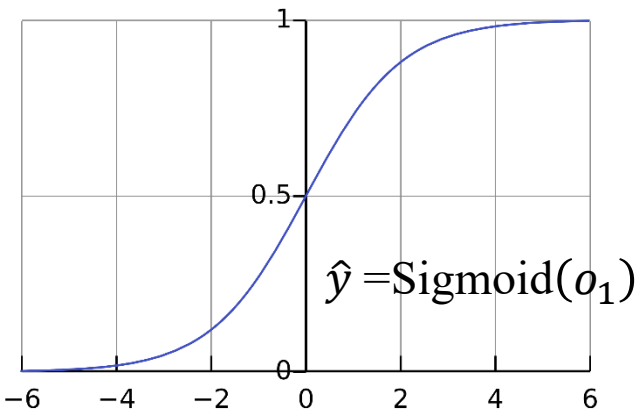
$$o_1 = \theta^T x \qquad \text{Sigmoid}(o_1) = \frac{1}{1+\exp(-o_1)}$$

$\hat{y} \geq 0.5$: Soluble; $\hat{y} < 0.5$: Non-soluble

Loss function: **Minimize** negative log likelihood:

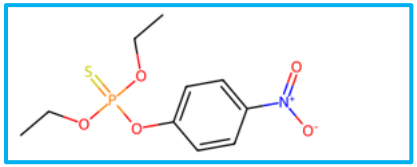
$$\ell_{MLE} = -\sum_{y_i=1} \log(\hat{\boldsymbol{y}}_i) - \sum_{y_i=0} \log(1-\hat{\boldsymbol{y}}_i)$$

$y_i = 1$, Class 1 $\quad y_i = 0$, Class 2



Linear Neural Networks for Classification

Softmax Regression for multi-class classification:



Task: 3-label classification: soluble, weakly soluble, non-soluble using 4 features (MW, Polar Surface Area, HBD, Rings):

e.g., Parathion: $(x_1, x_2, x_3, x_4) = (291.26, 70.83, 0, 1)$, weakly soluble.

One-hot encoding:

Soluble: (1, 0, 0)

Weakly soluble: (0, 1, 0)

Non-soluble: (0, 0, 1)

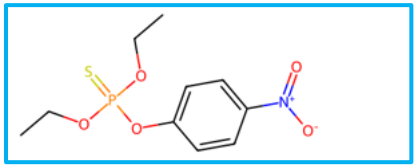
Labels: $y \in \{(1, 0, 0), (0, 1, 0), (0, 0, 1)\}$

Weights: $d = 4$ features, f =3 classes, 15 weights

$$\begin{cases} o_1 = x_1\theta_{11} + x_2\theta_{12} + x_3\theta_{13} + x_4\theta_{14} + \theta_{10} \\ o_2 = x_1\theta_{21} + x_2\theta_{22} + x_3\theta_{23} + x_4\theta_{24} + \theta_{20} \\ o_3 = x_1\theta_{31} + x_2\theta_{32} + x_3\theta_{33} + x_4\theta_{34} + \theta_{30} \end{cases}$$

Linear Neural Networks for Classification

Softmax Regression for multi-class classification:



Task: 3-label classification: soluble, weakly soluble, non-soluble using 4 features (MW, Polar Surface Area, HBD, # rings):

e.g., Parathion: $(x_1, x_2, x_3, x_4) = (291.26, 70.83, 0, 1)$, weakly soluble.

One-hot encoding:

Soluble: (1, 0, 0)

Weakly soluble: (0, 1, 0)

Non-soluble: (0, 0, 1)

Labels: $y \in \{(1, 0, 0), (0, 1, 0), (0, 0, 1)\}$

Weights ($\boldsymbol{\theta}$): d features, f classes, n chemical compounds:

$$\boxed{\boldsymbol{o} = \boldsymbol{\theta}^T \boldsymbol{x}}$$

$$\boldsymbol{\theta} = \begin{bmatrix} \theta_{10}, \theta_{20}, \dots, \theta_{f0} \\ \theta_{11}, \theta_{21} \dots, \theta_{f1} \\ \theta_{12}, \theta_{22} \dots, \theta_{f2} \\ \dots \\ \theta_{1d}, \theta_{2d} \dots, \theta_{fd} \end{bmatrix} \qquad \boldsymbol{x} = \begin{bmatrix} 1, 1, \dots, 1 \\ x_1^{(1)}, x_1^{(2)} \dots, x_1^{(n)} \\ x_2^{(1)}, x_2^{(2)} \dots, x_2^{(n)} \\ \dots \\ x_d^{(1)}, x_d^{(2)} \dots, x_d^{(n)} \end{bmatrix}$$

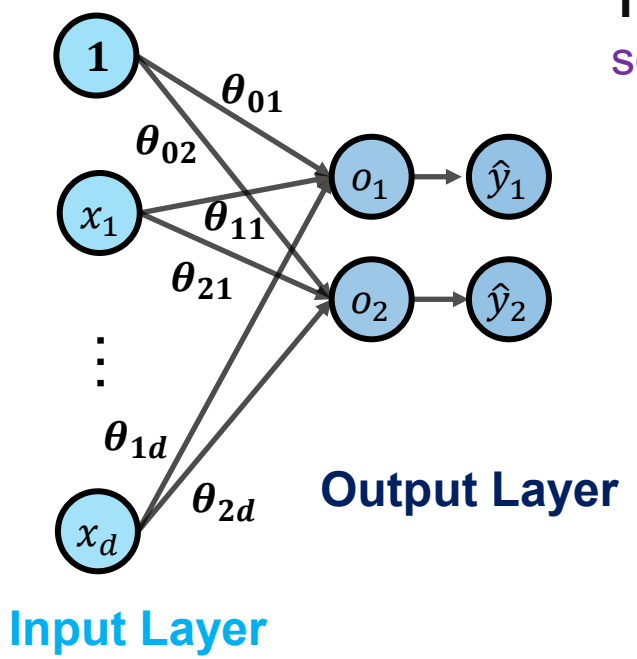
$\boldsymbol{o}$: $f \times n$ matrix

$(d+1) \times f$ matrix

$(d+1) \times n$ matrix

Linear Neural Networks for Classification

Softmax Regression for multi-class classification:



Task: 3-label classification: soluble, weakly soluble, non-soluble.

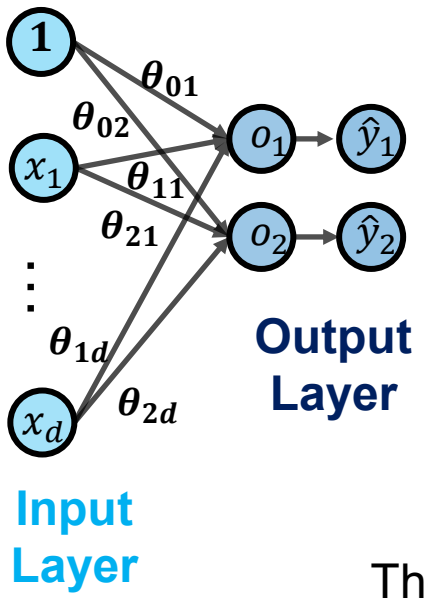
Weights: d features, f classes, n compounds:

$$\boldsymbol{o} = \boldsymbol{\theta}^{\boldsymbol{T}}\boldsymbol{x} \qquad \boldsymbol{o} = \begin{bmatrix} o_1^{(1)}, o_1^{(2)} \dots, o_1^{(n)} \\ o_2^{(1)}, o_2^{(2)} \dots, o_2^{(n)} \\ \dots \\ o_f^{(1)}, o_f^{(2)} \dots, o_f^{(n)} \end{bmatrix}$$

$\left\{o_1^{(j)}, \dots, o_f^{(j)}\right\}$ are not guaranteed to sum to 1 as expected for probabilities. They may be negative or larger than 1!

Linear Neural Networks for Classification

Softmax Regression for multi-class classification:



Task: 3-label classification: soluble, weakly soluble, non-soluble.

Weights: d features, f classes, n compounds:

$$\boldsymbol{o} = \boldsymbol{\theta}^{\boldsymbol{T}}\boldsymbol{x}$$

Softmax function: $\hat{\boldsymbol{y}} = \mathrm{Softmax}(\boldsymbol{o})$ $\quad \hat{y}_i^j = \dfrac{exp(o_i)}{\sum_{l=1}^{f} \exp(o_l)}$

The probability for the jth compound to be assigned to the ith class.

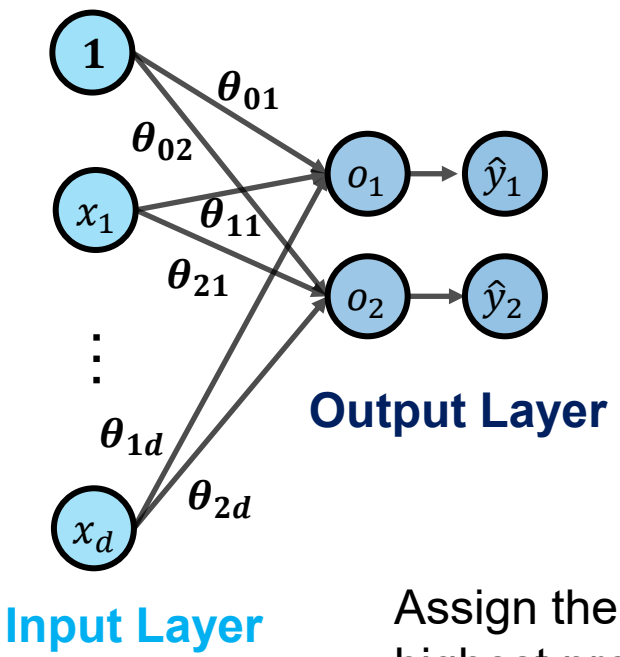
Physical Chemistry: Boltzmann distribution

$$P(E_i) = \frac{e^{-\beta E_i}}{\sum_i e^{-\beta E_i}}$$

The probability of the energy level E_i

Linear Neural Networks for Classification

Softmax Regression for multi-class classification:



Task: 3-label classification: soluble, weakly soluble, non-soluble.

Weights: d features, f classes, n compounds:

$$\boldsymbol{o} = \boldsymbol{\theta}^{\boldsymbol{T}}\boldsymbol{x}$$

Softmax function: $\hat{\boldsymbol{y}} = \mathrm{Softmax}(\boldsymbol{o})$ $\quad \hat{y}_i^j = \dfrac{exp(o_i)}{\sum_{l=1}^{f} \exp(o_l)}$

Assign the jth compound to its highest probability class: $\underset{i}{\mathrm{argmax}}\, \hat{y}_i$

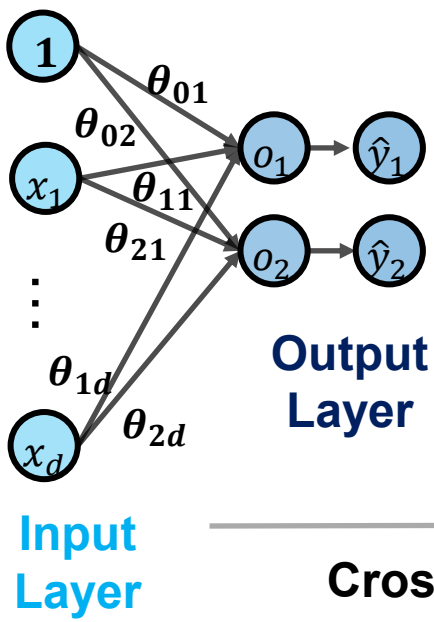
Example:

Log Solubility of Parathion: -3.95, $\boldsymbol{y} = (0,\ 1,\ 0)$ weakly soluble

Predicted: $\hat{\boldsymbol{y}} = (0.00,\ \ 0.94,\ \ 0.06)$ → assigned to the weakly soluble class.

Linear Neural Networks for Classification

Softmax Regression for multi-class classification:



Loss function:

Minimize negative log likelihood:

$$-\sum_{j=1}^{n} \log p\left(y^{(j)} | x^{(j)}\right)$$

Cross-entropy loss:

$$-\sum_{j=1}^{n} \sum_{i=1}^{f} y_i \log \hat{y}_i$$

Cross entropy between P and Q distributions:

$$\mathrm{H}(P, Q) = -\sum_i P_i \log Q_i$$

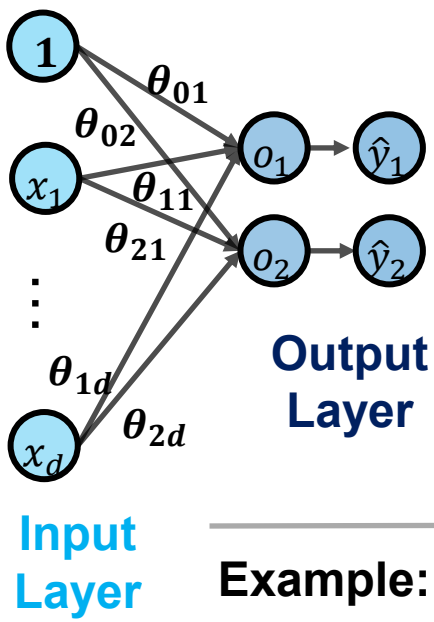
The expected "surprise" when data from true probabilities P is compared to subjective probabilities Q.

If we can't perfectly predict events, surprises occur — greater for lower-probability events.

$$-\log Q_i = \frac{1}{\log Q_i}$$

Linear Neural Networks for Classification

Softmax Regression for multi-class classification:



Loss function:

Minimize negative log likelihood:

$$-\sum_{j=1}^{n} \log p\left(y^{(j)} | x^{(j)}\right)$$

Cross-entropy loss:

$$-\sum_{j=1}^{n} \sum_{i=1}^{f} y_i \log \hat{y}_i$$

Model's likelihood of predicting the actual classes.

Example: $n = 1$

Parathion: $\hat{\mathbf{y}} = (0.00,\ 0.94,\ 0.06)$ $\mathbf{y} = (0, 1, 0)$ weakly soluble

Likelihood of the correct prediction: $\log p\left(y^{(j)} | x^{(j)}\right) = \log 0.94$

Cross-entropy: $n = 1$

$$-\sum_{j=1}^{n} \sum_{i=1}^{f} y_i \log \hat{y}_i = 0 \times 0 + 1 \times \log 0.94 + 0 \times log 0.06 = \log 0.94$$

Softmax and cross-entropy are commonly used in neural networks!

Softmax Regression: Example

Task: 3-label classification using 2 features (MW, Polar surface area):

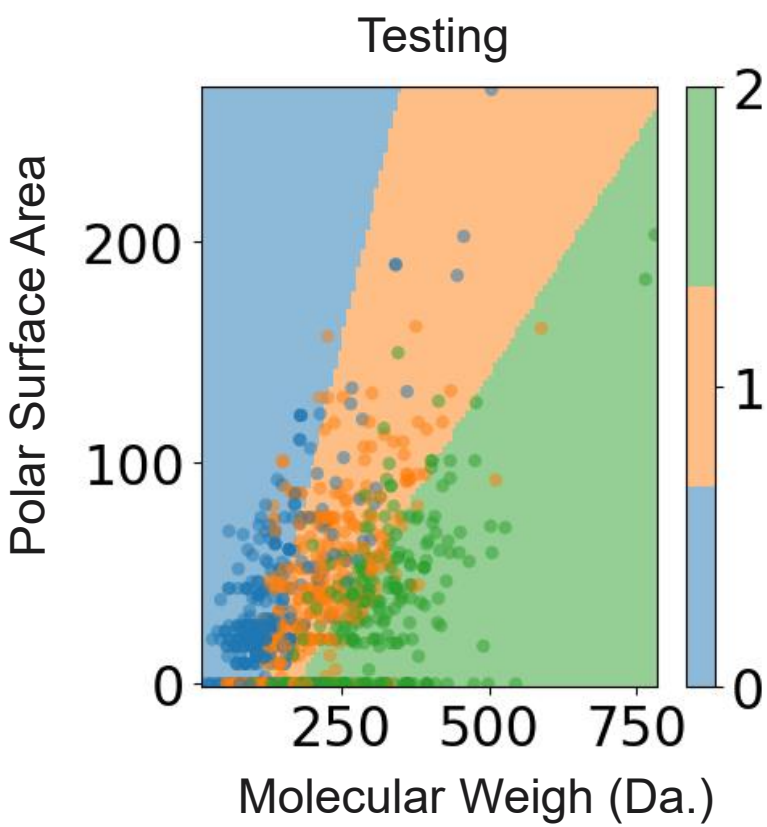
Soluble: $\mathrm{LogS} \geq -2$

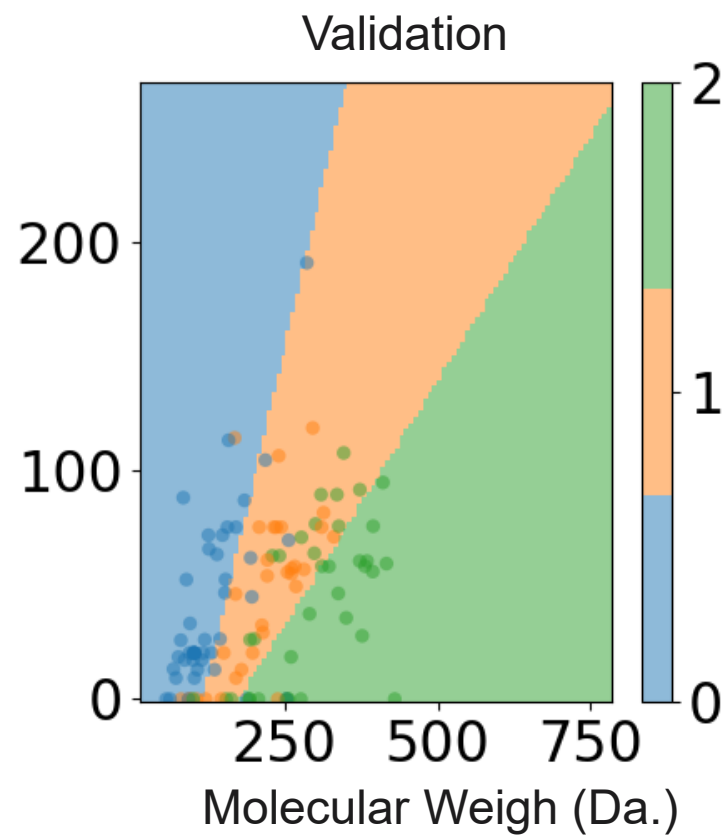
Weakly soluble: $-4 \leq \mathrm{LogS} < -2$

Non-soluble: $\mathrm{LogS} < -4$

1128 compounds

90/10 training/validation split.





Delaney Dataset. ***J. Chem. Inf. Comput. Sci.*** 44, 1000, (2004)

Softmax Regression Example

Task: 3-label classification using 4 features (MW, Hdonor, HAcceptor, Rings):

Soluble: $\mathrm{LogS} \geq -2$

Weakly soluble: $-4 \leq \mathrm{LogS} < -2$

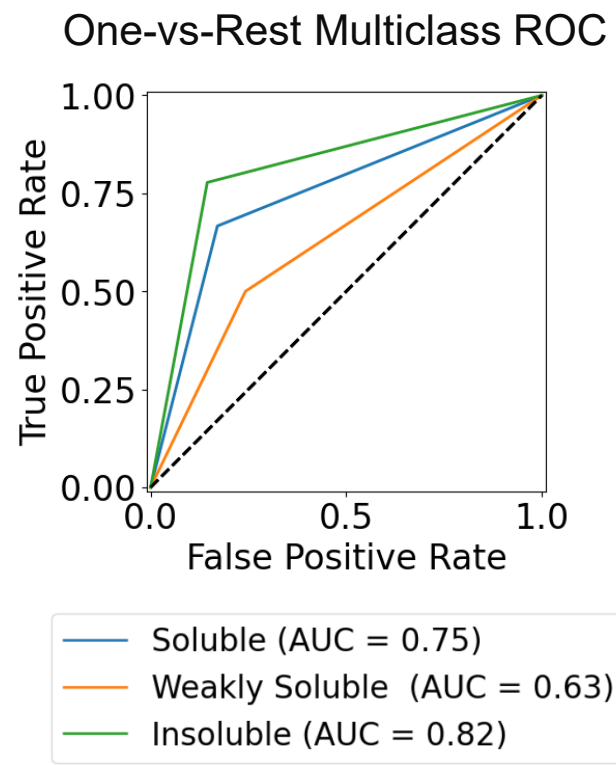
Non-soluble: $\mathrm{LogS} < -4$

1128 compounds

90/10 training/validation split.

Evaluation on the Validation Set (112 compounds)

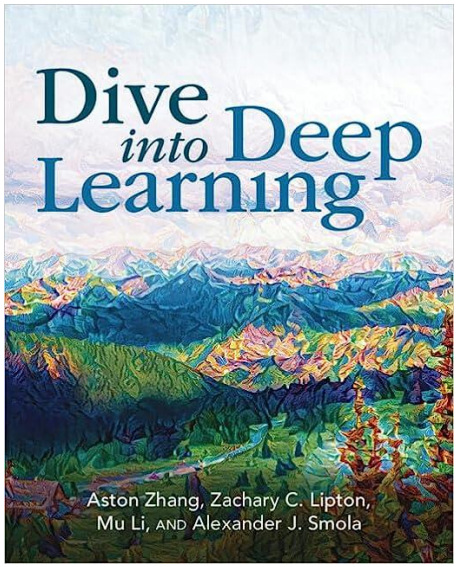
Confusion Matrix		Predicted		
		Soluble	Weakly Soluble	Insoluble
Actual	Soluble	20	9	1
	Weakly Soluble	13	23	10
	Insoluble	1	7	28



Outline

- Linear Models as Neural Networks
- **Multilayer Perceptron (MLP): A Nonlinear Classifier**
  - **Activation Functions**
  - Forward Propagation
  - Backward Propagation
- Gradient Vanishing or Exploding
- Techniques to Prevent Overfitting

Reading material:



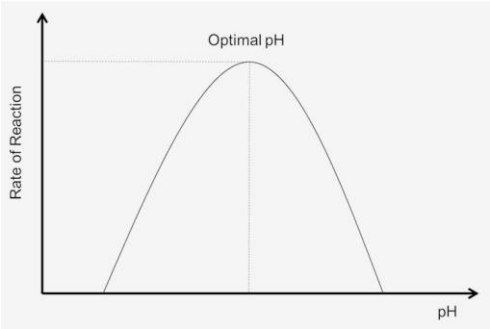
Chapter 5

https://d2l.ai/

Limitations of Linear Neural Networks

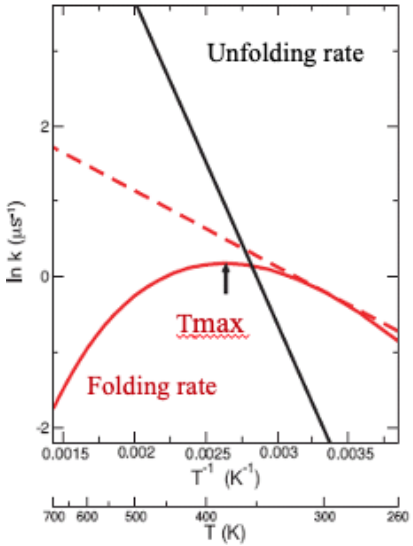
Simple linear or monotonic models cannot fully describe complex chemical behaviors:

- **pH & enzyme activity** follows a bell-shaped curve.
- **Protein folding rate** as a function of temperate is non-linear due to entropic free energy barriers.
- **Solubility v.s. temperature** often shows non-monotonic solubility trends, with solubility increasing up to a point and then leveling off.
- **Chemical yield optimization**: The effect of reagent concentrations on product yield is often non-linear due to competing side reactions.



Enzyme activity v.s. pH

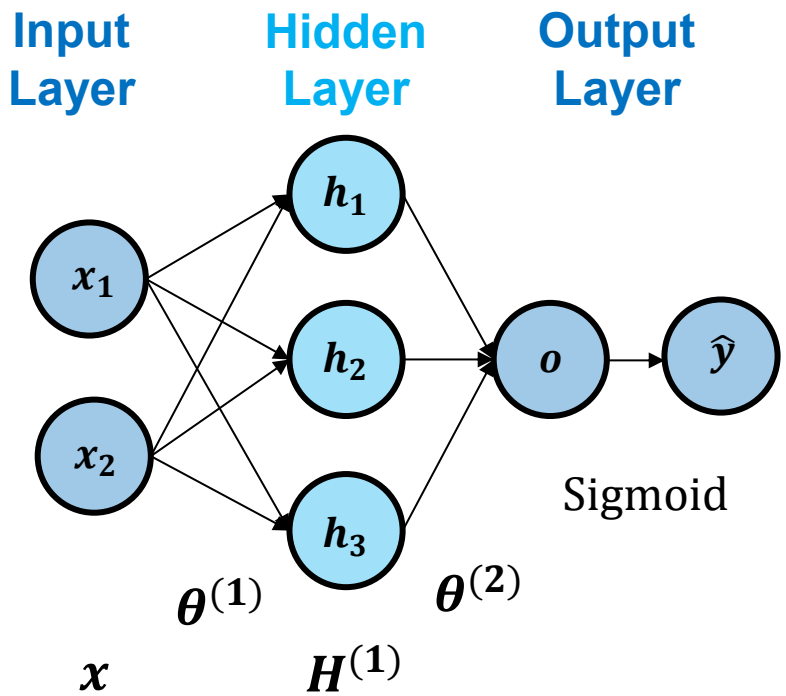
https://www.creative-enzymes.com/



Non-Arrhenius protein folding

PNAS (2007) 104:15340

From Linear to Nonlinear: Hidden Layers



Multilayer Perceptrons (MLP): A fully connected neural network with one or more hidden layers.

- Input layer to Hidden layer:

$$\boldsymbol{H}^{(1)} = \sigma(\boldsymbol{\theta}^{(1)T}\boldsymbol{x})$$

σ: activation function.

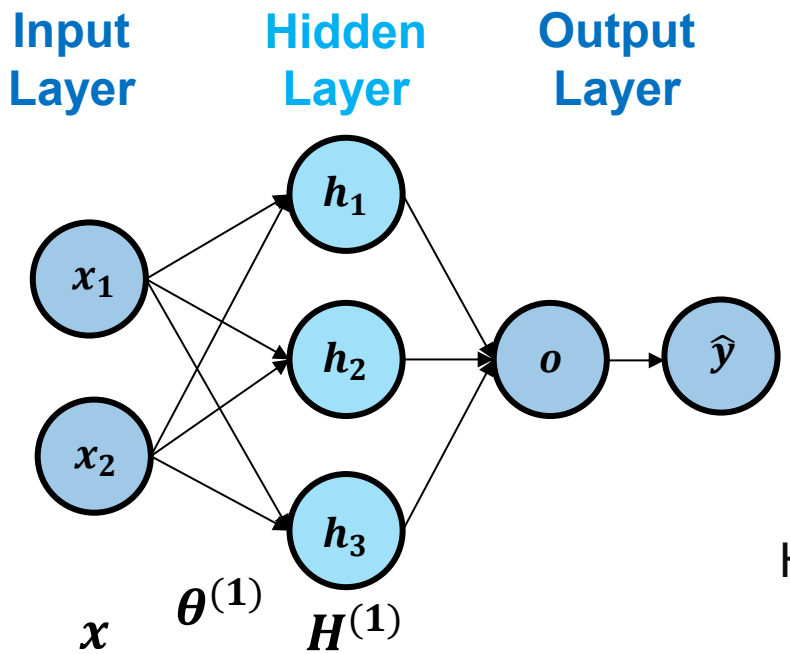
- Hidden layer to Output layer:

$$\boldsymbol{o} = \boldsymbol{\theta}^{(2)T}\boldsymbol{H}^{(1)}$$

Binary classification: $\hat{\boldsymbol{y}} = \mathrm{Sigmoid}(\boldsymbol{o})$

- More hidden layers: $\boldsymbol{H}^{(1)}$, $\boldsymbol{H}^{(2)}$, … - MLP can go deep!
- Use activation functions to learn non-linear relationships in chemical data.
- Softmax function for multiple-label classification.

From Linear to Nonlinear: Activation Functions

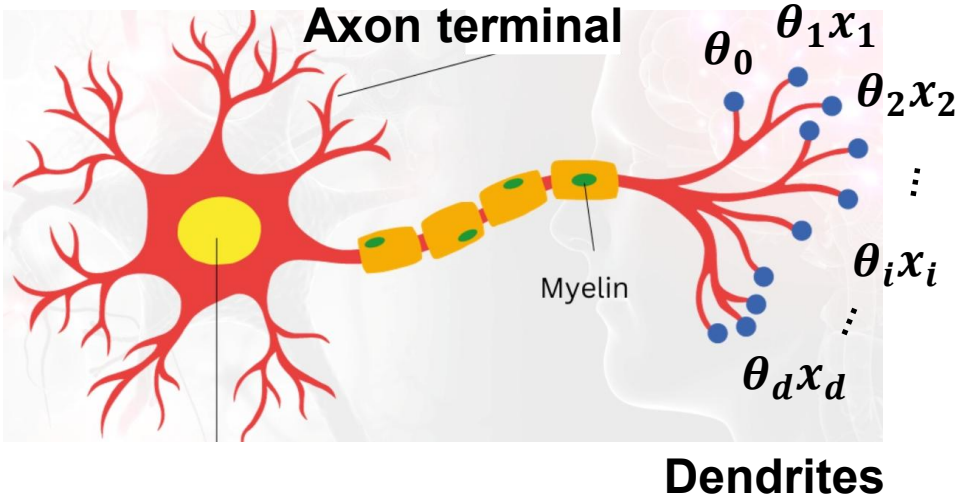


MLP: non-linear activation function on hidden layers introduces non-linearity to the model.

$$H^{(1)} = \sigma(\theta^{(1)T} x)$$

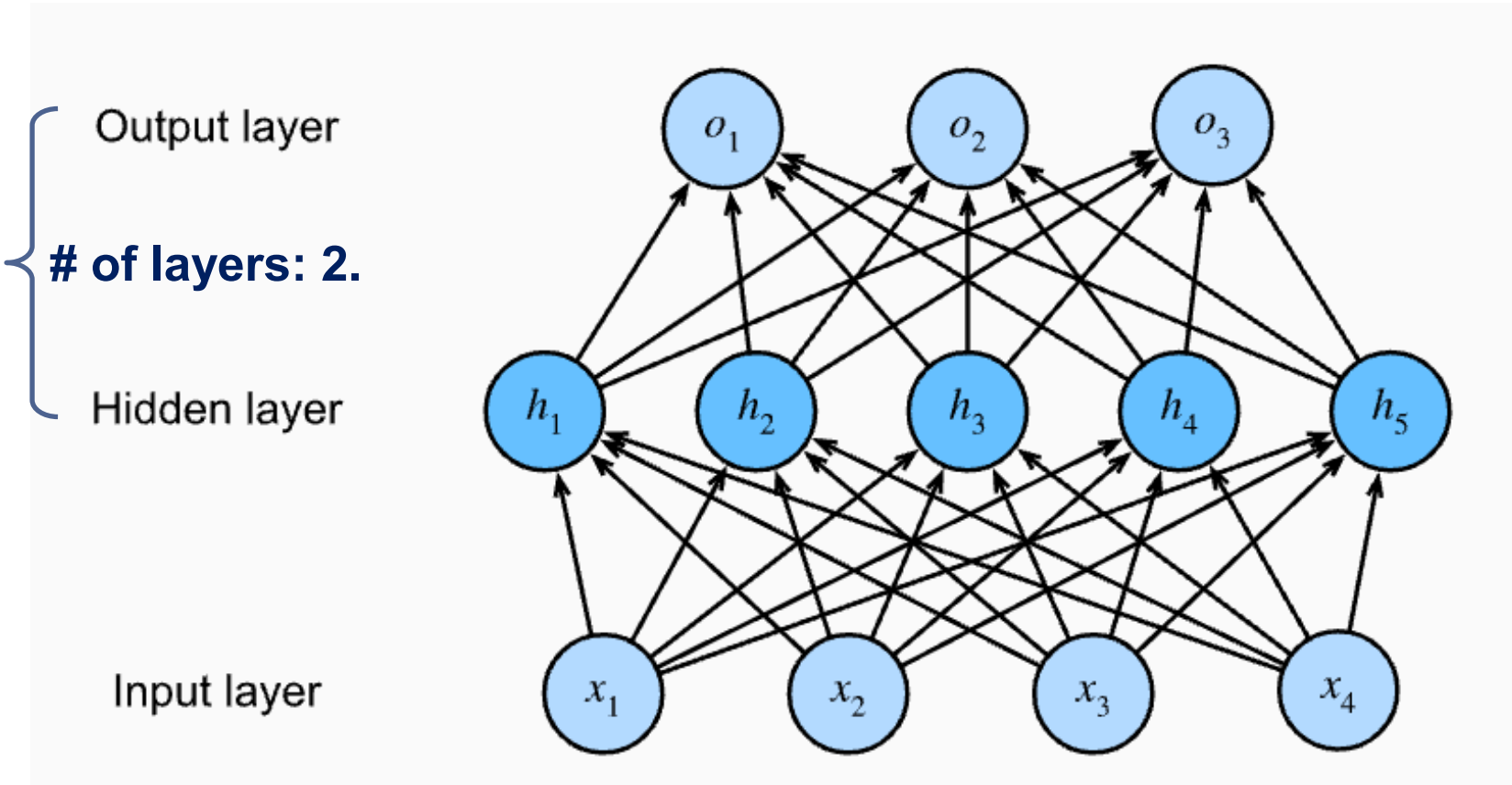
σ: activation function.

Hidden layer contains hidden neurons.



Axon applies a non-linear activation function σ to its input.

From Linear to Nonlinear: Activation Functions



Input layer does not involve calculations.

https://d2l.ai/

From Linear to Nonlinear: Activation Functions

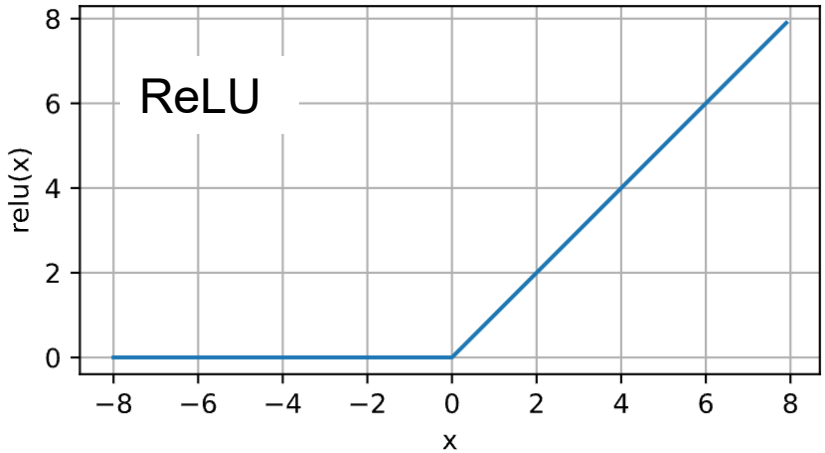
MLP: non-linear activation function on hidden layers introduces non-linearity to the model.

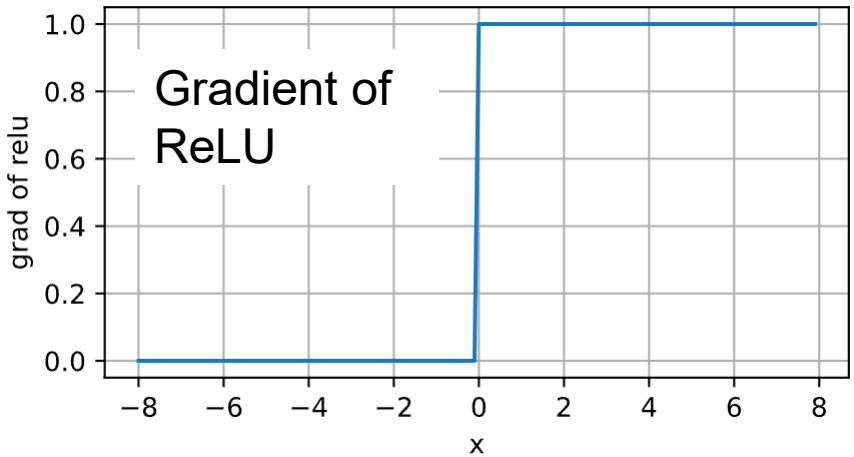
$$\boldsymbol{H}^{(1)} = \sigma(\boldsymbol{\theta}^{(1)T}\boldsymbol{x})$$

σ: activation function.

Common choices: (2) Rectified linear unit (**ReLU**):

$$ReLU(x) = \max(0, x)$$





Its derivatives are simple: either vanish or pass the input through. Widely used in hidden layers

https://d2l.ai/

From Linear to Nonlinear: Activation Functions

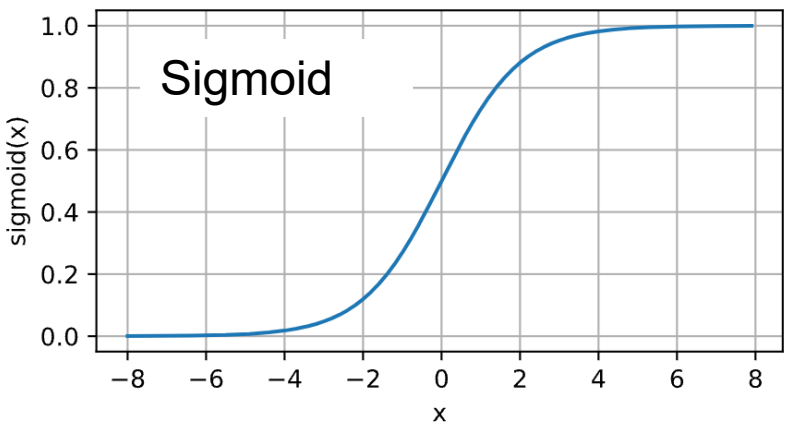
MLP: non-linear activation function on hidden layers introduces non-linearity to the model.

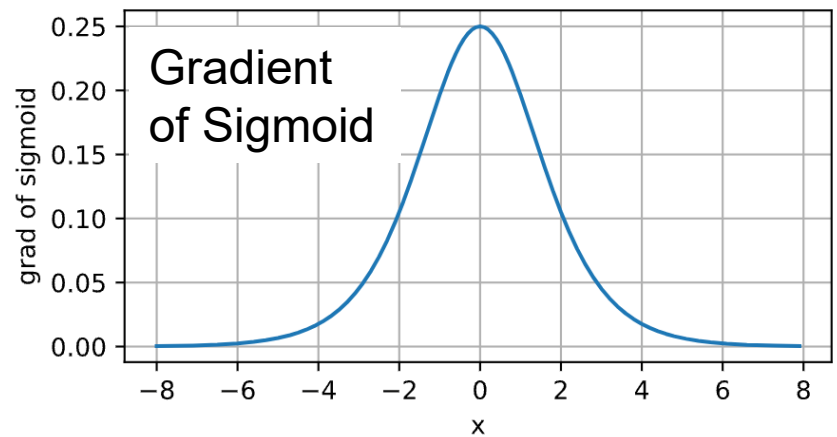
$$\boldsymbol{H}^{(1)} = \sigma(\boldsymbol{\theta}^{(1)T}\boldsymbol{x})$$

σ: activation function.

Common choices: (2) **Sigmoid**

$$Sigmoid(x) = \frac{1}{1+e^{-x}}$$





Pro: Good for binary classification, as it outputs probabilities.
Con: Its gradients vanish for large inputs, making optimization challenging and limiting its use in hidden layers.

https://d2l.ai/

From Linear to Nonlinear: Activation Functions

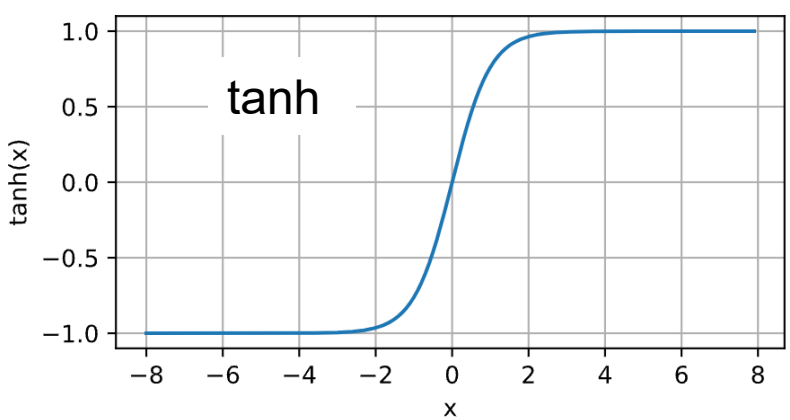
MLP: non-linear activation function on hidden layers introduces non-linearity to the model.

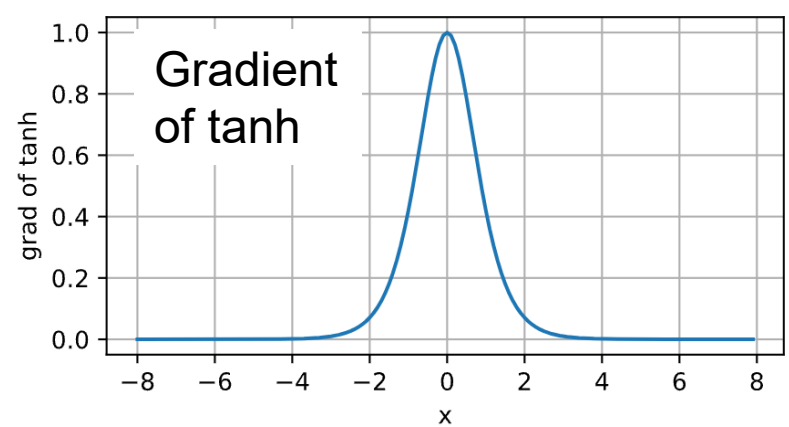
$$\boldsymbol{H}^{(1)} = \sigma(\boldsymbol{\theta}^{(1)T}\boldsymbol{x})$$

σ: activation function.

Common choices: (3) **hyperbolic tangent:**

$$tanh(x) = \frac{1 - e^{-2x}}{1 + e^{-2x}}$$



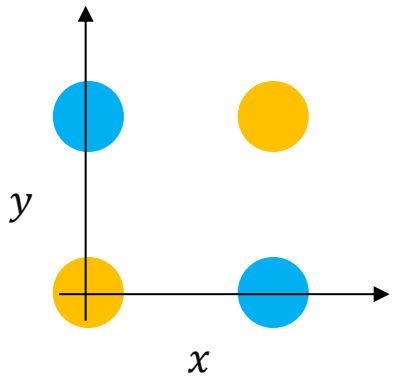


Pro: outputs values between -1 and 1 (point symmetric to “0”).

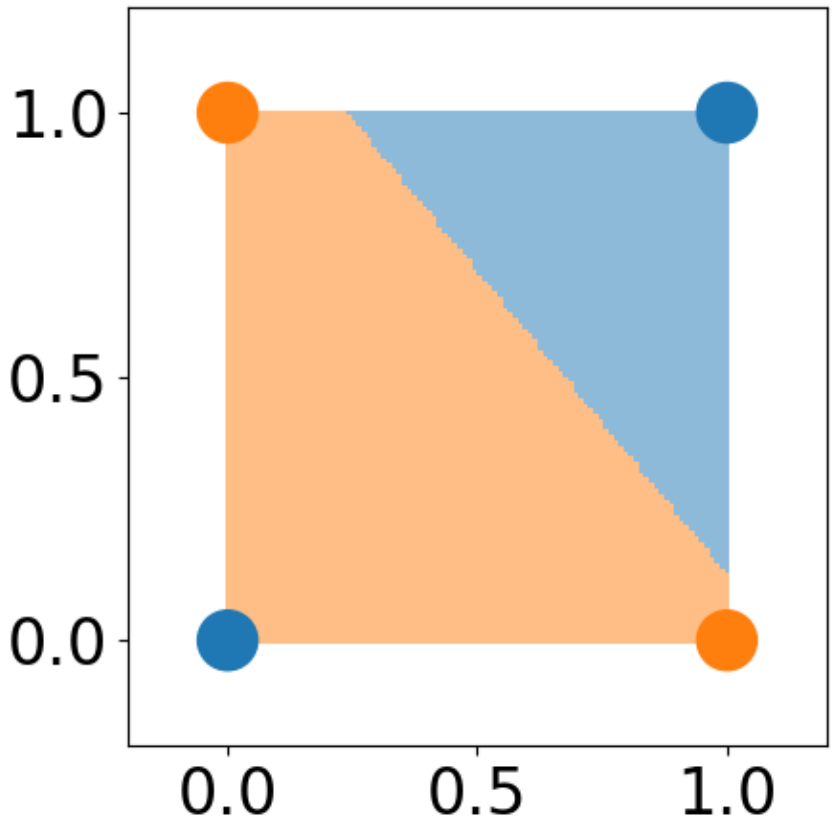
Con: Similar with Sigmoid, its gradients vanish for large inputs.

From Linear to Nonlinear: Simple Example

Task: Training a model to predict the labels (blue v.s. yellow) of a dataset of 4 points.



Logistic Regression linear neural networks can only reach the accuracy of 75%.



Linear decision boundary

4 points

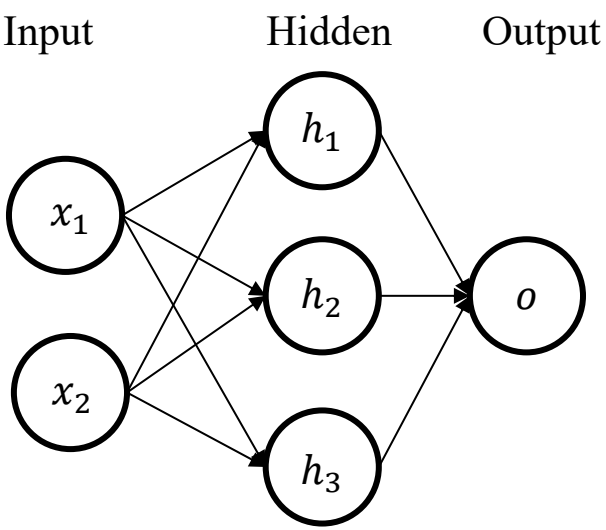
$$\boldsymbol{X} = \begin{bmatrix} 0 & 0 \\ 0 & 1 \\ 1 & 0 \\ 1 & 1 \end{bmatrix}$$

True Labels

$$\boldsymbol{Y} = \begin{bmatrix} 0 \\ 1 \\ 1 \\ 0 \end{bmatrix}$$

From Linear to Nonlinear: Simple Example

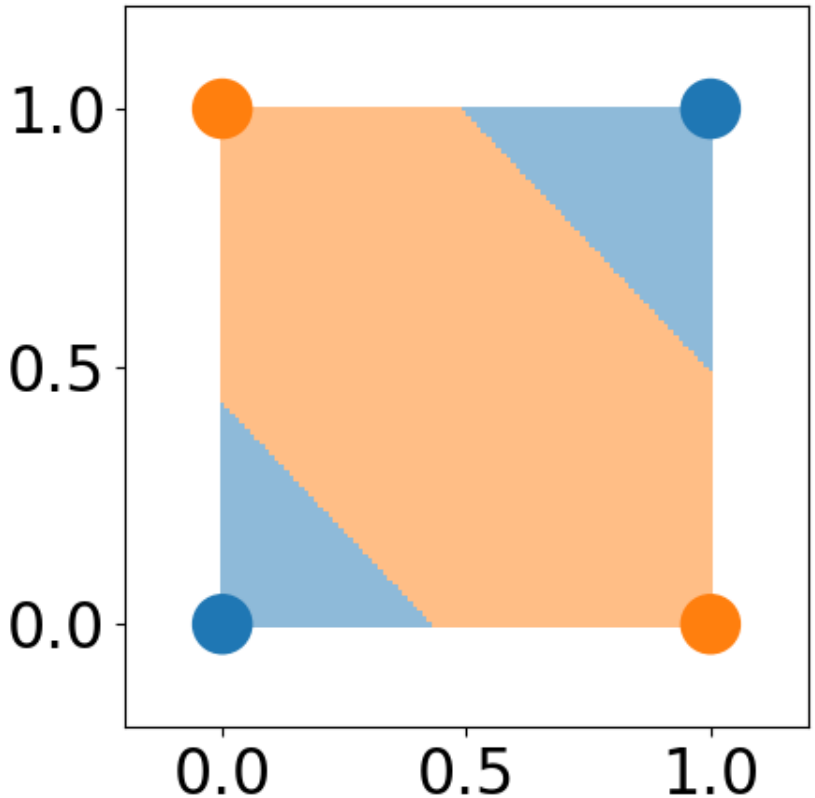
Task: Training a model to predict the labels (blue v.s. yellow) of a dataset of 4 points.



$$\hat{y} = \text{sigmoid}(o)$$
$$= \text{sigmoid}(\text{ReLU}(XW^{(1)T})W^{(2)T})$$

Hidden layers + non-linear activation function!

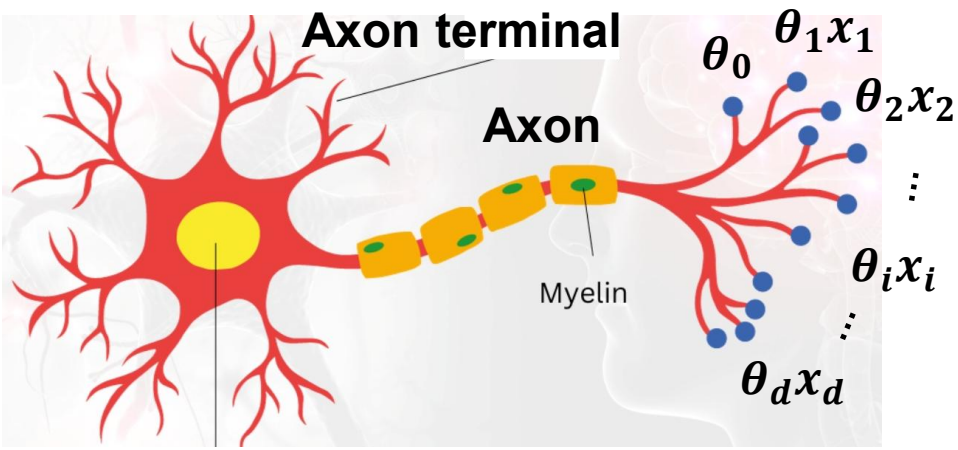
The MLP can perfectly separate the two classes of data points.



Review of the Previous Lecture

Linear models as single-layer neural networks:

A neuron:



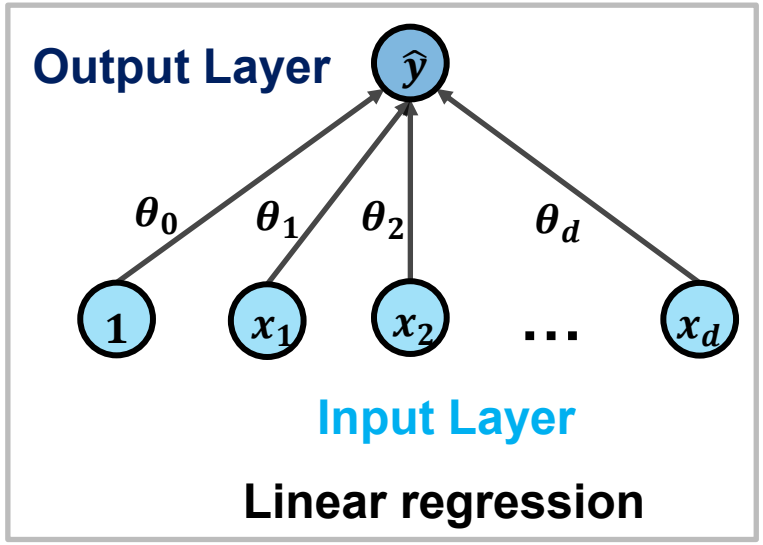
https://formalpsychology.com/

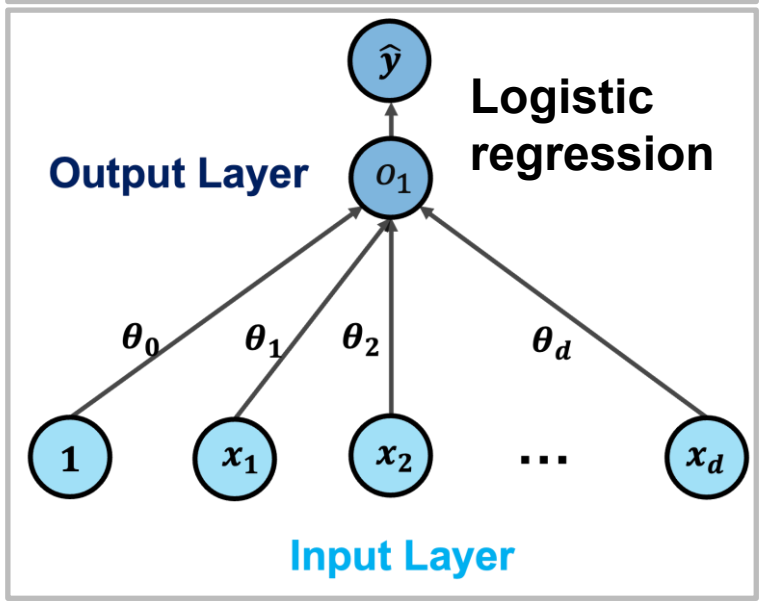
Dendrites

Dendrites: total input: weighted sum from d resources: $\hat{y} = \theta_0 \cdot 1 + \cdots \theta_d x_d$

Axon post-processes by applying an activation function $\sigma(\hat{y})$.

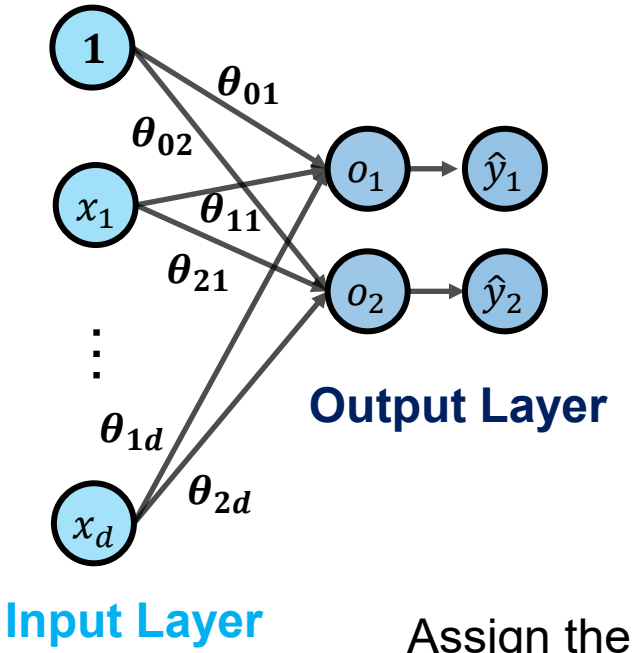
Axon terminal transmits the output to its destination.





Review of the Previous Lecture

Softmax Regression (linear model) for multi-class classification:



$$\boldsymbol{o} = \boldsymbol{\theta}^T \boldsymbol{x} \qquad \boldsymbol{o} = \begin{bmatrix} o_1^{(1)}, o_1^{(2)} \dots, o_1^{(n)} \\ o_2^{(1)}, o_2^{(2)} \dots, o_2^{(n)} \\ \dots \\ o_f^{(1)}, o_f^{(2)} \dots, o_f^{(n)} \end{bmatrix}$$

Softmax function:

$$\hat{\boldsymbol{y}} = \mathrm{Softmax}(\boldsymbol{o}) \qquad \hat{y}_i^j = \frac{exp(o_i)}{\sum_{l=1}^{f} \exp(o_l)}$$

Assign the jth compound to its highest probability class: $\underset{i}{\mathrm{argmax}}\, \hat{y}_i$

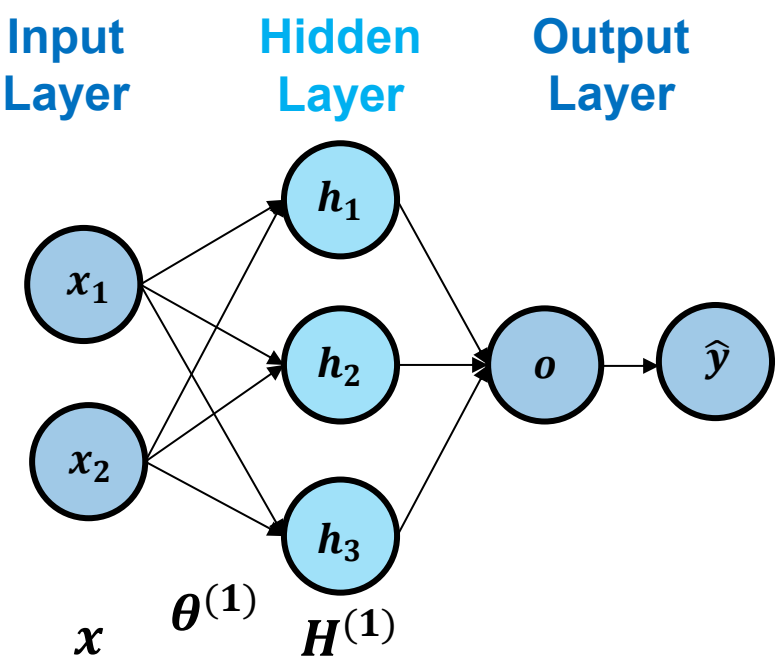
Example:

Log Solubility of Parathion: -3.95, $\boldsymbol{y} = (0,\ 1,\ 0)$ weakly soluble

Predicted: $\hat{\boldsymbol{y}} = (0.00,\ \ 0.94,\ \ 0.06)$ → assigned to the weakly soluble class.

Review of the Previous Lecture

Multilayer Perceptrons (MLP): A fully connected neural network with one or more hidden layers.



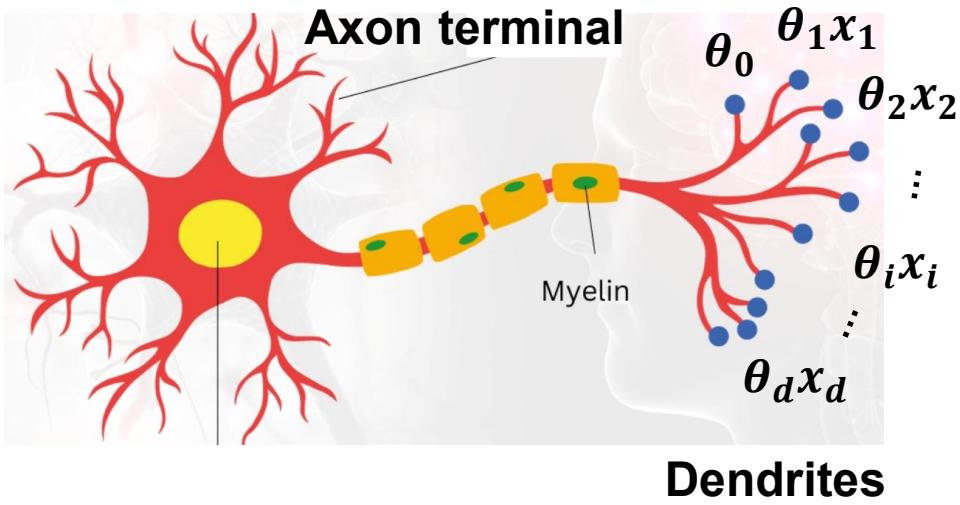
Non-linear activation function on hidden layers introduces non-linearity to the model.

$$H^{(1)} = \sigma(\theta^{(1)T} x)$$

σ: activation function.

Hidden layer contains hidden neurons.

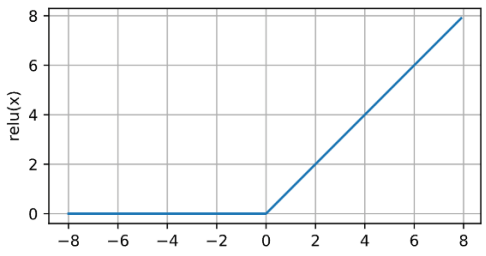
Axon applies a non-linear activation function σ to its input.

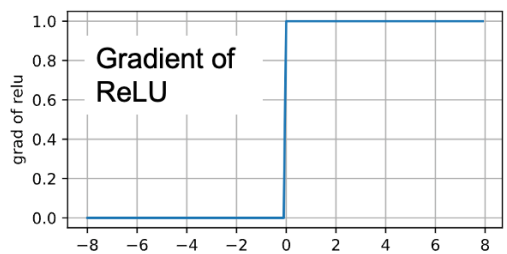


Review of the Previous Lecture

Non-linear activation function on hidden layers introduces non-linearity to the model.

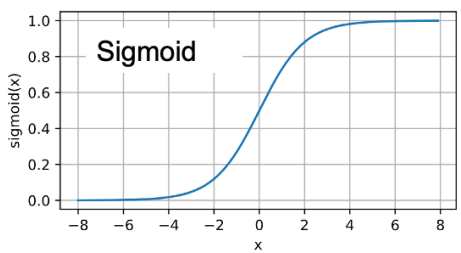
(1) Rectified linear unit (**ReLU**): $ReLU(x) = \max(0, x)$

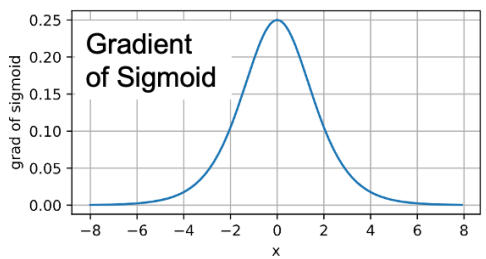




Widely used in hidden layers!

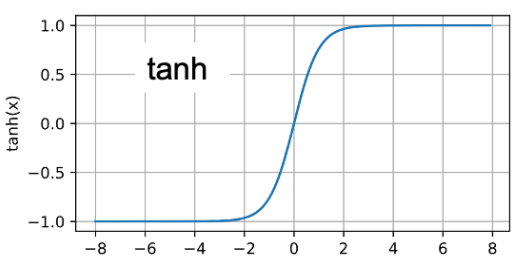
(2) **Sigmoid** $Sigmoid(x) = \dfrac{1}{1 + e^{-x}}$

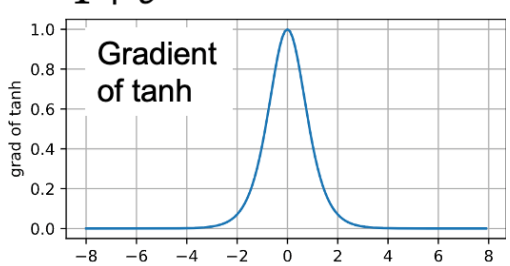




Gradients vanish for large inputs, making the gradient descent optimization challenging.

(3) **hyperbolic tangent:** $tanh(x) = \dfrac{1 - e^{-2x}}{1 + e^{-2x}}$

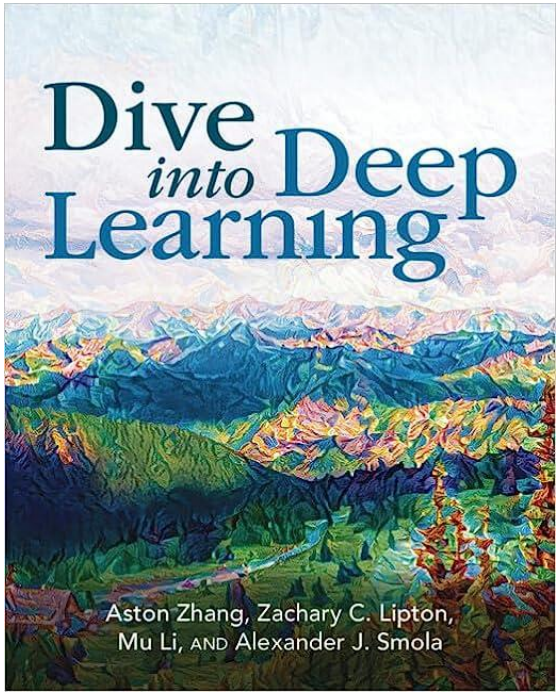




Gradients vanish for large inputs.

Review of the Previous Lecture

Reading material:



Chapter 5

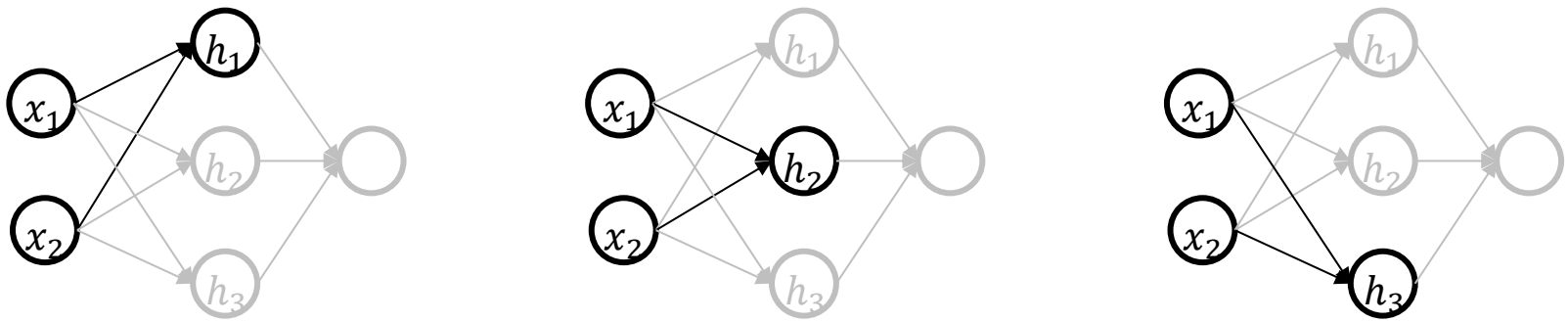
https://d2l.ai/

Outline

- Linear Models as Neural Networks
- Multilayer Perceptron (MLP): A Nonlinear Classifier
  - Activation Functions
  - **Forward Propagation**
  - Backward Propagation
- Gradient Vanishing or Exploding
- Techniques to Prevent Overfitting

Forward Propagation

Forward propagation calculates outputs and intermediate variables as data moves through a neural network.



$$h_1^{(1)} = \theta_{11}^{(1)} x_1 + \theta_{12}^{(1)} x_2 + \theta_{10}^{(1)} \qquad h_2^{(1)} = \theta_{21}^{(1)} x_1 + \theta_{22}^{(1)} x_2 + \theta_{20}^{(1)} \qquad h_3^{(1)} = \theta_{31}^{(1)} x_1 + \theta_{32}^{(1)} x_2 + \theta_{30}^{(1)}$$

Matrix notation:

$$\boldsymbol{H}^{(1)} = \sigma(\boldsymbol{z}), \quad \boldsymbol{z} = \boldsymbol{\theta}^{(1)T} \boldsymbol{x} = \boxed{\boldsymbol{X}\boldsymbol{W}^{(1)T} + \boldsymbol{b}^{(1)}}$$

We switch to common notations in ML (pytorch):

$$w_{ij}^{(1)} = \theta_{ij}^{(1)}$$

$$b_i^{(1)} = \theta_{i0}^{(1)}$$

i-th Layer

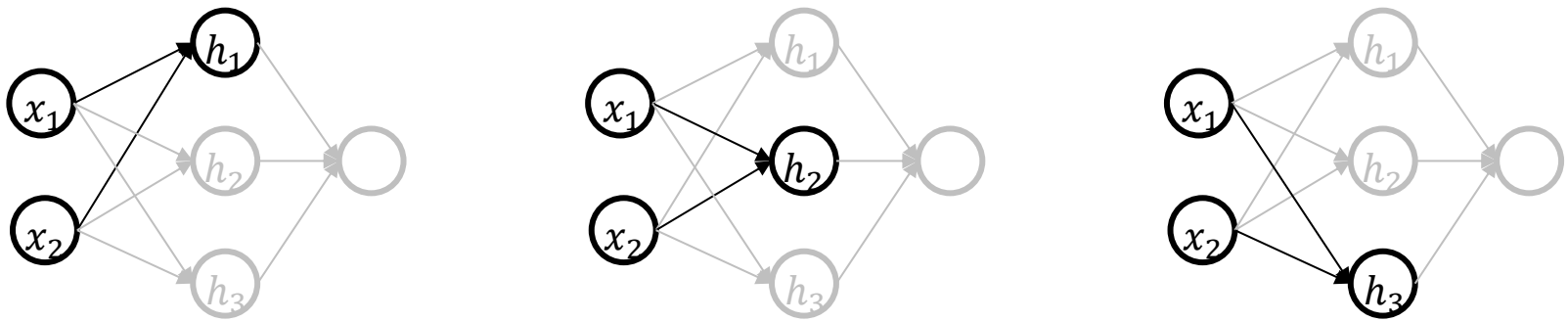
Weights → $w_{12}^{(1)}$

Output index

Input index

Forward Propagation

Forward propagation calculates outputs and intermediate variables as data moves through a neural network.



Matrix notation:

$$\boldsymbol{H}^{(1)} = \sigma\left(\boldsymbol{z}^{(1)}\right), \quad \boldsymbol{z}^{(1)} = \boldsymbol{\theta}^{(1)T}\boldsymbol{x} = \boxed{\boldsymbol{X}\boldsymbol{W}^{(1)T} + \boldsymbol{b}^{(1)}}$$

$$\begin{bmatrix} z_1^{(1)} & z_2^{(1)} & z_3^{(1)} \end{bmatrix} = \begin{bmatrix} x_1 & x_2 \end{bmatrix} \begin{bmatrix} w_{11}^{(1)} & w_{12}^{(1)} \\ w_{21}^{(1)} & w_{22}^{(1)} \\ w_{31}^{(1)} & w_{32}^{(1)} \end{bmatrix}^T + \begin{bmatrix} b_1^{(1)} & b_2^{(1)} & b_3^{(1)} \end{bmatrix}$$

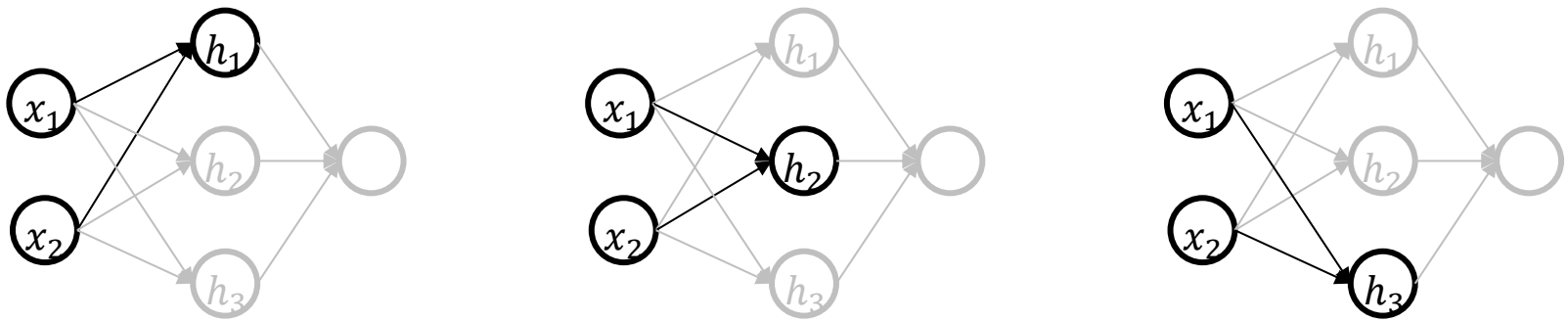
$\mathrm{n} \times 3$ matrix
n: # of data points.
$\mathrm{n} = 1$

$\mathrm{n} \times 2$ matrix
$\mathrm{n} = 1$

3×2 matrix

Forward Propagation

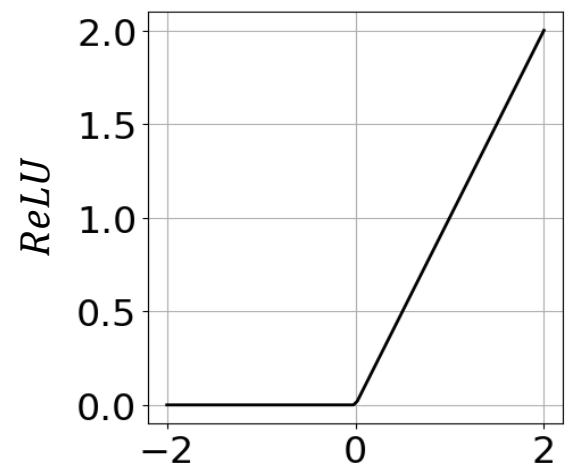
Forward propagation calculates outputs and intermediate variables as data moves through a neural network.



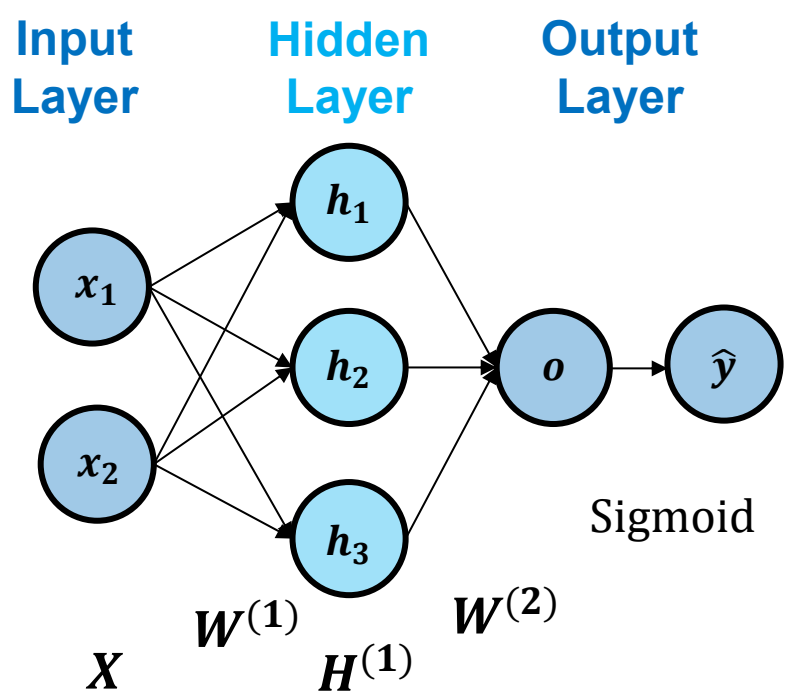
Matrix notation: $\boldsymbol{z}^{(1)} = \boldsymbol{X}\boldsymbol{W}^{(1)T} + \boldsymbol{b}^{(1)}$

Non-linear activation by ReLU: $\boldsymbol{H}^{(1)} = ReLU\left(\boldsymbol{z}^{(1)}\right)$

$$\begin{bmatrix} h_1^{(1)} & h_2^{(1)} & h_3^{(1)} \end{bmatrix} = ReLU\left(\begin{bmatrix} z_1^{(1)} & z_2^{(1)} & z_3^{(1)} \end{bmatrix}\right)$$



Forward Propagation



Data flow in MLP:

1. Input layer to Hidden layer:

$$\boldsymbol{H}^{(1)} = ReLU(\boldsymbol{X}\boldsymbol{W}^{(1)T} + \boldsymbol{b}^{(1)})$$

2. Hidden layer to Output layer:

$$\boldsymbol{o} = \boldsymbol{H}^{(1)}\boldsymbol{W}^{(2)T} + \boldsymbol{b}^{(2)}$$

$$\hat{\boldsymbol{y}} = \text{Sigmoid}(\boldsymbol{o})$$

3. Compute loss function for binary classification:

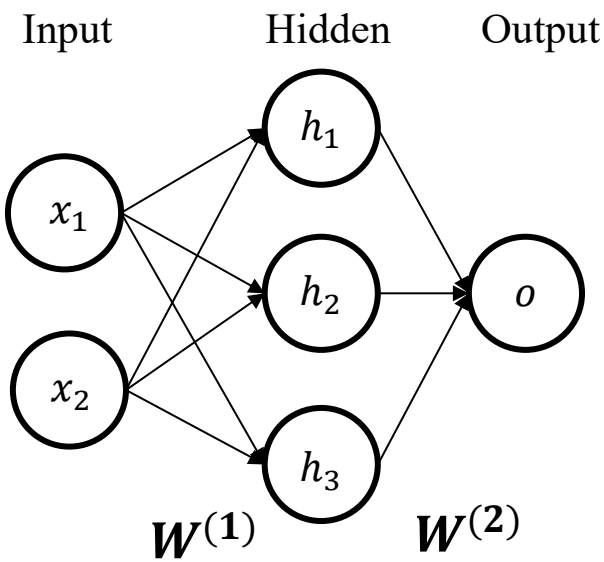
$$\ell = -\sum_{y_i=1} \log(\hat{\boldsymbol{y}}_i) - \sum_{y_i=0} \log(1 - \hat{\boldsymbol{y}}_i)$$

4. Gradient descent on the loss function to update weights.

How to compute the gradient of the loss function? – Backpropagation!

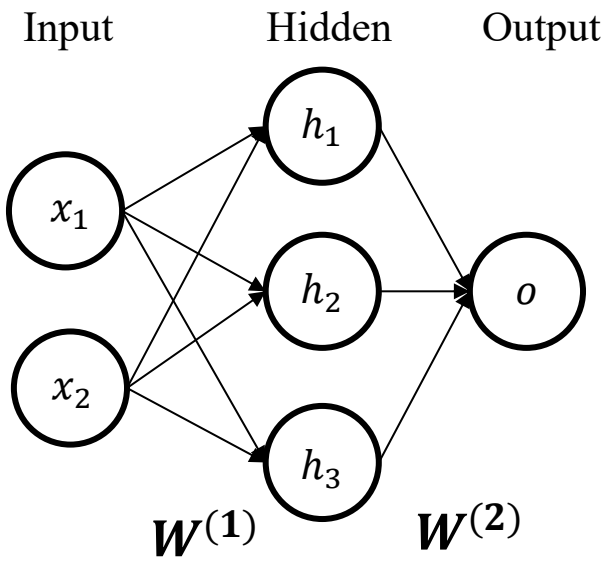
Exercise

How many weight parameters does this MLP have?



Exercise

How many weight parameters does this MLP have?



$\boldsymbol{W}^{(1)}$ is a 3×2 matrix, containing 6 parameters

$\boldsymbol{W}^{(2)}$ is a 1×3 matrix, containing 3 parameters

$\boldsymbol{b}^{(1)}$: 3 parameters

$\boldsymbol{b}^{(2)}$: 1 parameter

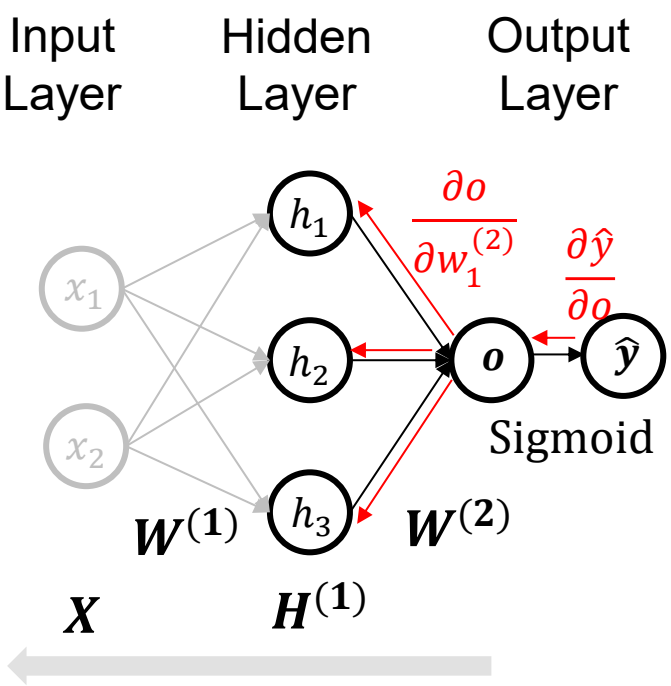
Total: 13 parameters

Outline

- Linear Models as Neural Networks
- Multilayer Perceptron (MLP): A Nonlinear Classifier
  - Activation Functions
  - Forward Propagation
  - **Backward Propagation**
- Gradient Vanishing or Exploding
- Techniques to Prevent Overfitting

Backpropagation

Backpropagation calculates gradients of neural network parameters by moving backward through the network using the chain rule.



Apply chain rule in reverse to compute partial derivatives on all weights: $\boldsymbol{W}^{(1)}$ & $\boldsymbol{W}^{(2)}$.

Gradient descent: $w_{t+1} \leftarrow w_t - \alpha \cdot \nabla_w \ell(\hat{\boldsymbol{y}})$

Step 1: $\hat{y}(o) \rightarrow o$ $\left(\frac{\partial \ell(\hat{y})}{\partial \hat{y}}\right)\left(\frac{\partial \hat{y}}{\partial o}\right)$

Step 2: $o \rightarrow$ Hidden layer

$$\mathrm{o} = h_1^{(1)} w_1^{(2)} + h_2^{(1)} w_2^{(2)} + h_3^{(1)} w_3^{(2)} + b^{(2)}$$

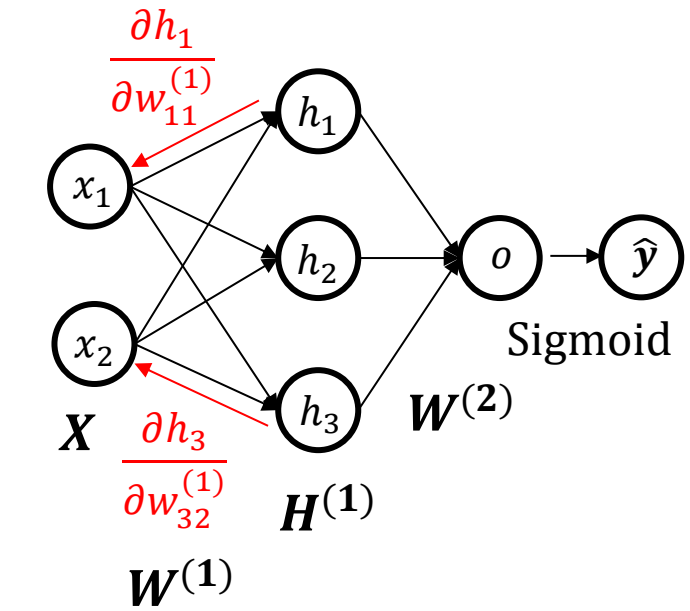
$$\left(\frac{\partial o}{\partial w_1^{(2)}}\right) = h_1^{(1)}, \ldots, \left(\frac{\partial o}{\partial b^{(2)}}\right) = 1$$

Derivatives on $W^{(1)}$:

$$\frac{\partial \ell(\hat{y})}{\partial w_1^{(2)}} = \left(\frac{\partial \ell(\hat{y})}{\partial \hat{y}}\right)\left(\frac{\partial \hat{y}}{\partial o}\right)\left(\frac{\partial o}{\partial w_1^{(2)}}\right)$$

Backpropagation

Backpropagation calculates gradients of neural network parameters by moving backward through the network using the chain rule.

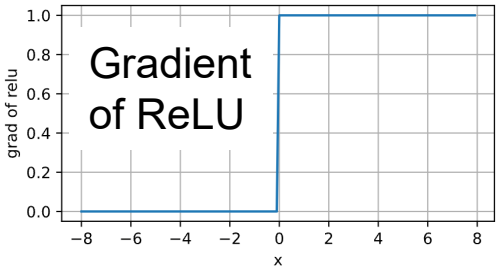


Gradient descent: $w_{t+1} \leftarrow w_t - \alpha \cdot \nabla_w \ell(\hat{\boldsymbol{y}})$

Step 3: Hidden layer → Input layer

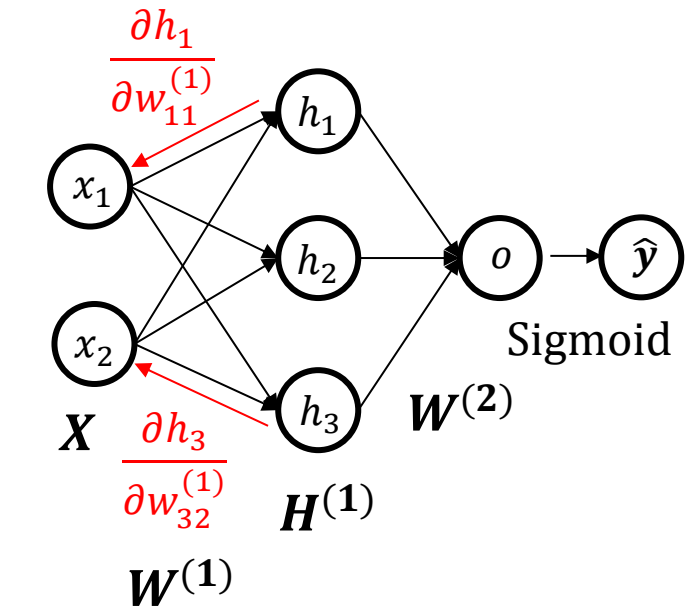
$$\boldsymbol{H}^{(1)} = ReLU(\boldsymbol{X}\boldsymbol{W}^{(1)T} + \boldsymbol{b}^{(1)})$$

$$\frac{\partial h_1^{(1)}}{\partial w_{11}^{(1)}} = ReLU'\left(h_1^{(1)}\right) x_1$$



Backpropagation

Backpropagation calculates gradients of neural network parameters by moving backward through the network using the chain rule.

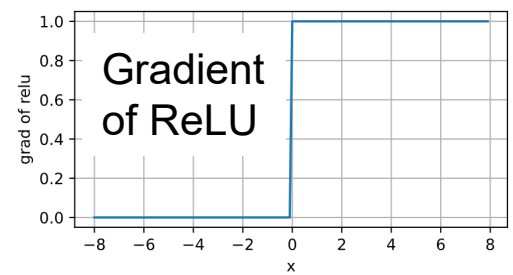


Gradient descent: $w_{t+1} \leftarrow w_t - \alpha \cdot \nabla_w \ell(\hat{\boldsymbol{y}})$

Step 3: Hidden layer → Input layer

$$\boldsymbol{H}^{(1)} = ReLU(\boldsymbol{X}\boldsymbol{W}^{(1)T} + \boldsymbol{b}^{(1)})$$

$$\frac{\partial h_1^{(1)}}{\partial w_{11}^{(1)}} = ReLU'\left(h_1^{(1)}\right) x_1$$



Derivatives on $W^{(1)}$:

$$\frac{\partial \ell(\hat{y})}{\partial w_{11}^{(1)}} = \frac{\partial \ell(\hat{y})}{\partial o} \cdot \frac{\partial o}{\partial h_1^{(1)}} \cdot \frac{\partial h_1^{(1)}}{\partial w_{11}^{(1)}}$$

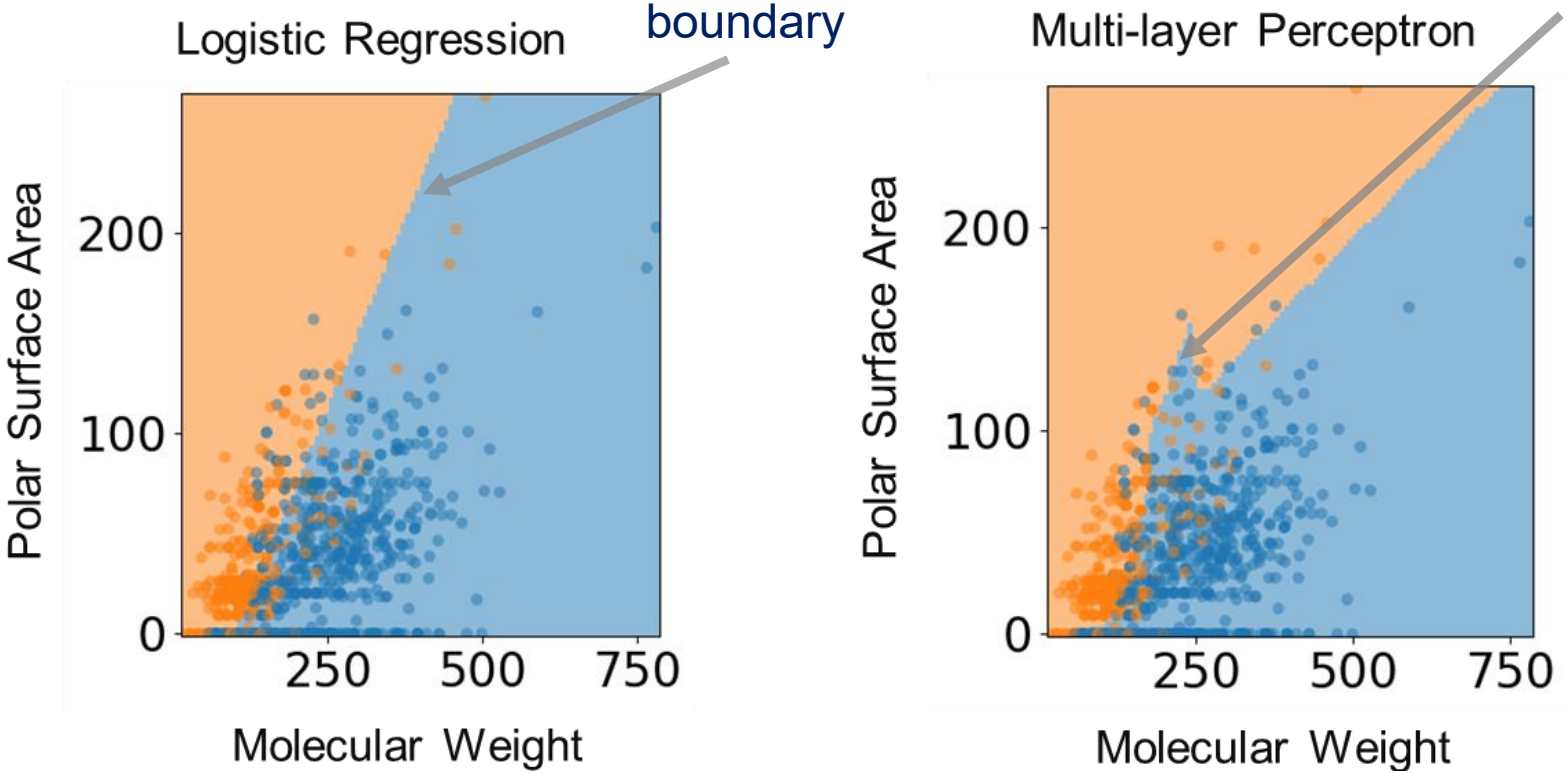
MLP: Example

Task: Training a MLP model to predict soluble v.s. non-soluble compounds.

Soluble: $\mathrm{LogS} \geq -2$; Weakly soluble: $\mathrm{LogS} < -2$

Non-linear decision boundary is more accurate on training data!

Linear decision boundary

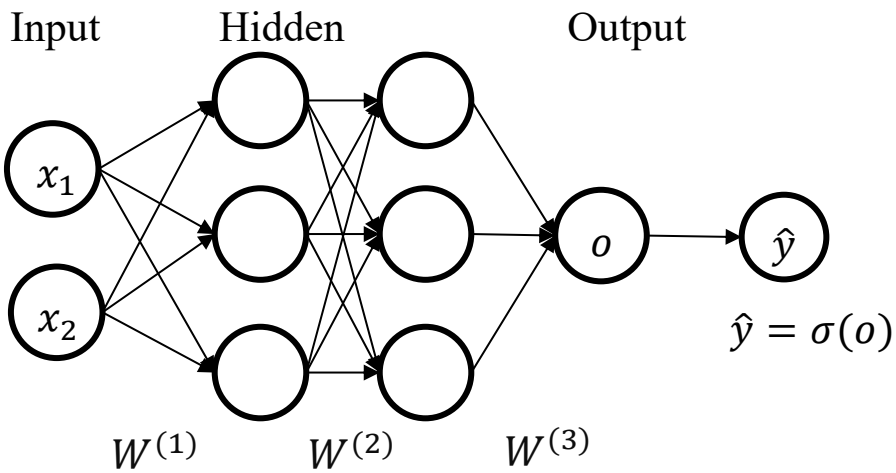


1128 compounds

Delaney Dataset. ***J. Chem. Inf. Comput. Sci.*** 44, 1000, (2004)

Exercise

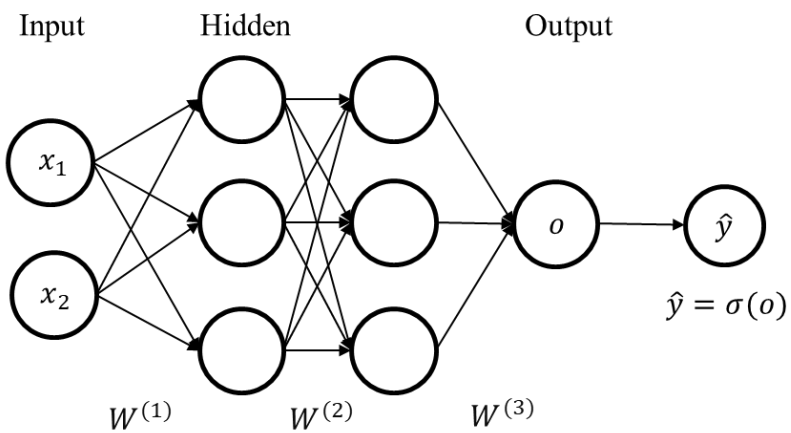
For the gradients of $W^{(1)}$ in the neural networks on the right, could you match the two options below with the ReLU and Sigmoid activation functions?

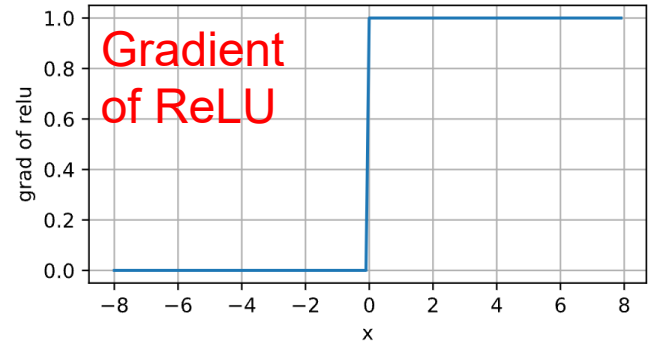


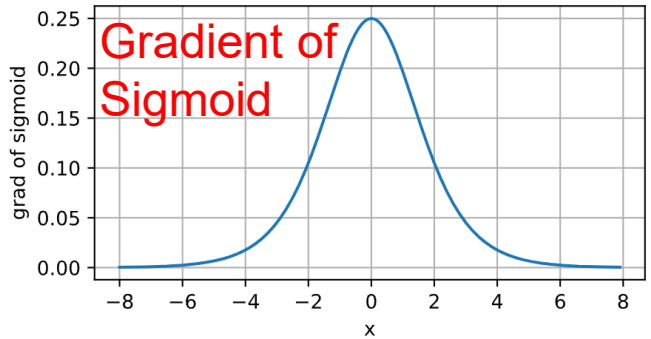
(a): $\nabla W^{(1)} = \begin{bmatrix} 0.0011 & 0.0002 \\ 0.0083 & -0.0126 \\ 0.0101 & -0.0044 \end{bmatrix}$

(b): $\nabla W^{(1)} = \begin{bmatrix} 1.0744\text{e}-03 & -6.9180\text{e}-05 \\ 2.4076\text{e}-04 & -3.8335\text{e}-05 \\ -9.3095\text{e}-04 & 2.0548\text{e}-04 \end{bmatrix}$

Exercise







$$(\mathrm{A}): \nabla W^{(1)} = \begin{bmatrix} 0.0011 & 0.0002 \\ 0.0083 & -0.0126 \\ 0.0101 & -0.0044 \end{bmatrix}$$

ReLU

$$(\mathrm{B}): \nabla W^{(1)} = \begin{bmatrix} 1.0744\mathrm{e}{-03} & -6.9180\mathrm{e}{-05} \\ 2.4076\mathrm{e}{-04} & -3.8335\mathrm{e}{-05} \\ -9.3095\mathrm{e}{-04} & 2.0548\mathrm{e}{-04} \end{bmatrix}$$

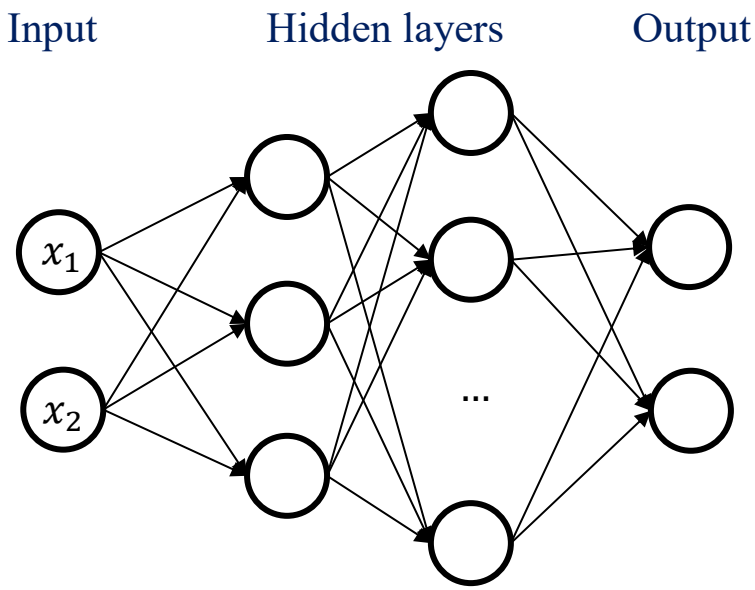
Sigmoid

Sigmoid/Tanh function saturate, causing derivatives to be small (vanishing gradient). Even though *ReLU* may suffer from "dead neurons" where gradients become zero, but its gradient equals 1, as long as the neuron is not inactive.

Outline

- Linear Models as Neural Networks
- Multilayer Perceptron (MLP): A Nonlinear Classifier
  - Activation Functions
  - Forward Propagation
  - Backward Propagation
- **Gradient Vanishing or Exploding**
- Techniques to Prevent Overfitting

Gradient Vanishing or Exploding



During backpropagation, gradients can become vanishing (too small) or exploding (too large), disrupting training.

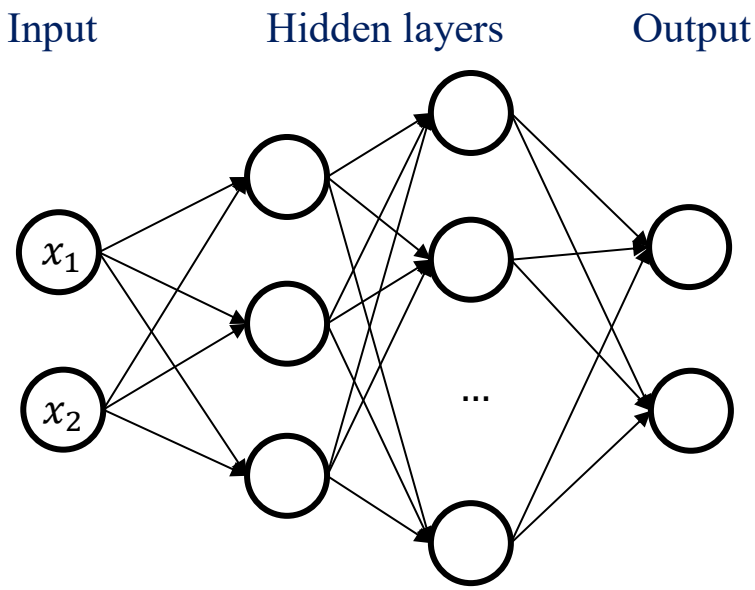
Gradient vanishing:

- Gradients shrink as they move backward, causing earlier layers to learn very slowly.
- Common in deep neural networks with **sigmoid** or **tanh** activations.

Gradient Exploding:

- Gradients grow exponentially, leading to unstable weight updates and training divergence.
- Often occurs with **poor weight initialization**.

Gradient Vanishing or Exploding



Impact on chemical data analysis:

- In molecular property prediction, vanishing gradients slow learning, while exploding gradients cause instability.
- Difficult to learn long-range correlations in quantum chemistry or graph-based models.

Solutions:

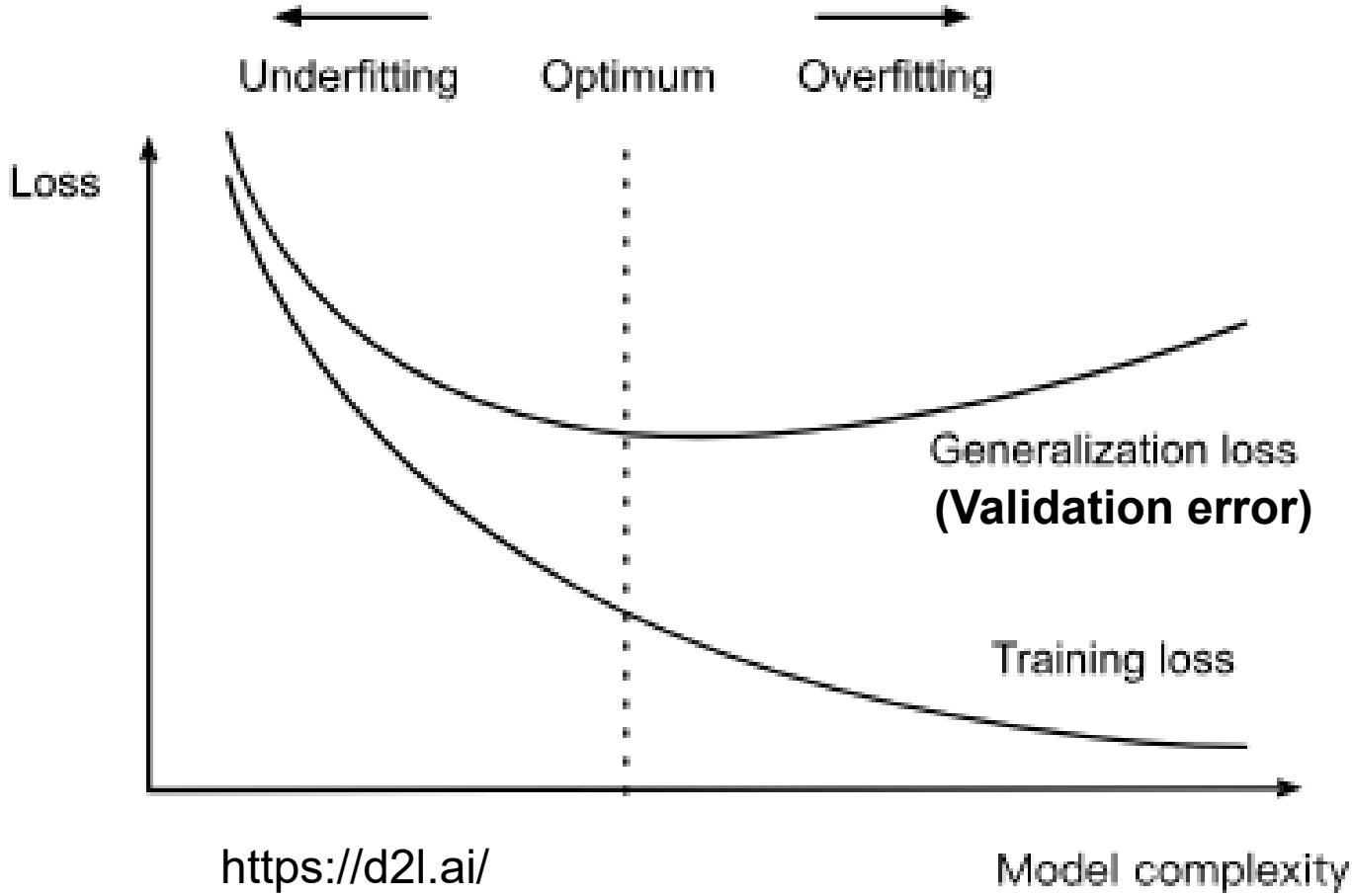
- Use ReLU activation to reduce vanishing gradients.
- Apply batch normalization (normalizes the output of each layer in the network by adjusting and scaling it) for more stable training.
- Gradient clipping (when the gradient exceeds a certain threshold, we scale it down to a predefined limit) to control exploding gradients.
- Better weight initialization.

Outline

- Linear Models as Neural Networks
- Multilayer Perceptron (MLP): A Nonlinear Classifier
  - Activation Functions
  - Forward Propagation
  - Backward Propagation
- Gradient Vanishing or Exploding
- **Techniques to Prevent Overfitting**

Overfitting

Overfitting: The training error is substantially lower than the generalization/validation error.



https://d2l.ai/

Techniques to Mitigate Overfitting

- **Regularization:**
  - Ridge regularization (L2 norm): $\ell'(\hat{\boldsymbol{y}}) = \ell(\hat{\boldsymbol{y}}) + \lambda||\boldsymbol{W}||_2^2$
  - Lasso regularization (L1 norm): $\ell'(\hat{\boldsymbol{y}}) = \ell(\hat{\boldsymbol{y}}) + \lambda||\boldsymbol{W}||_1$
- **Early stopping**: stop when the validation loss starts to increase (indicating overfitting).
- **Dropout**: random units (neurons) are ignored during training.
- **Data Augmentation**: increase the size of the training dataset.
- **Reduce model complexity**: Use simpler models (e.g., fewer layers) when the dataset is small or not complex enough to require a deep network.

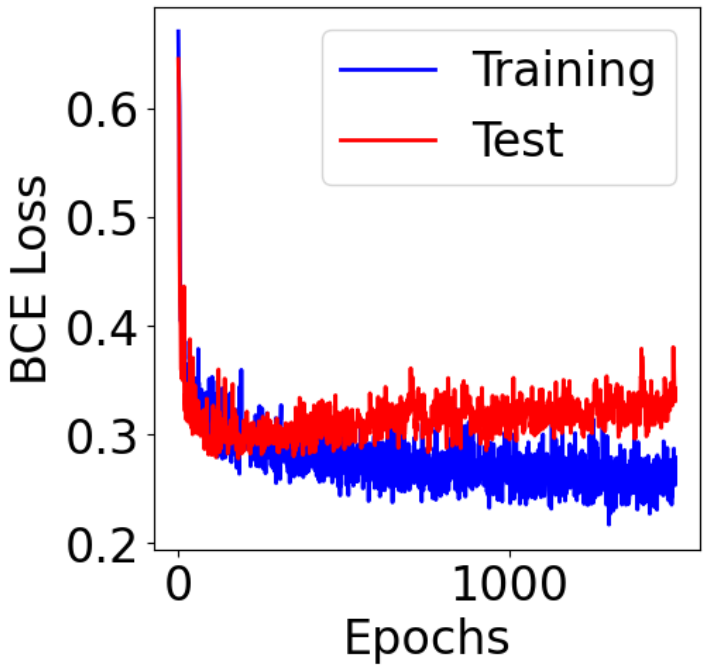
Regularization

Ridge regularization adds the L2 norm of the weight as a penalty to the loss function.

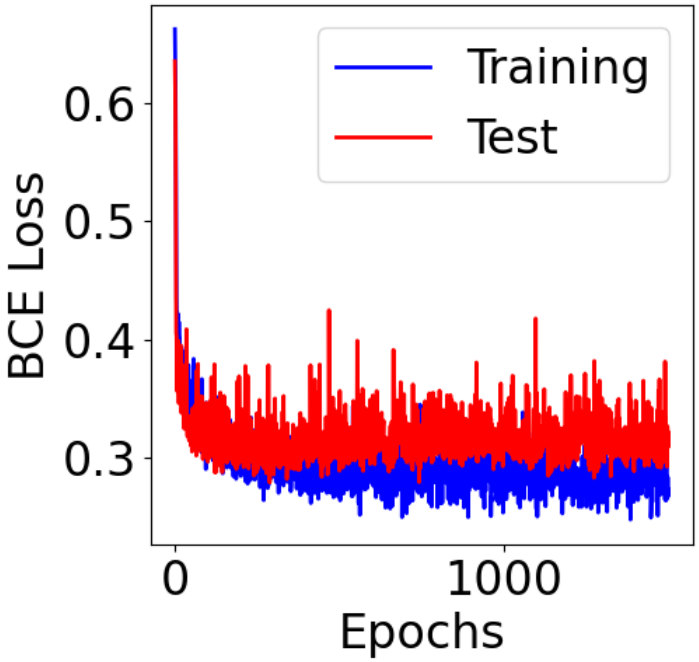
$$\ell'(\hat{\boldsymbol{y}}) = \ell(\hat{\boldsymbol{y}}) + \lambda||\boldsymbol{W}||_2^2 \qquad \theta_{t+1} \leftarrow \theta_t - \alpha\nabla_\theta \ell(f_\theta) - 2\alpha\lambda\theta$$

Predicting solubility based on 4 features:

No regulation



Ridge regularization



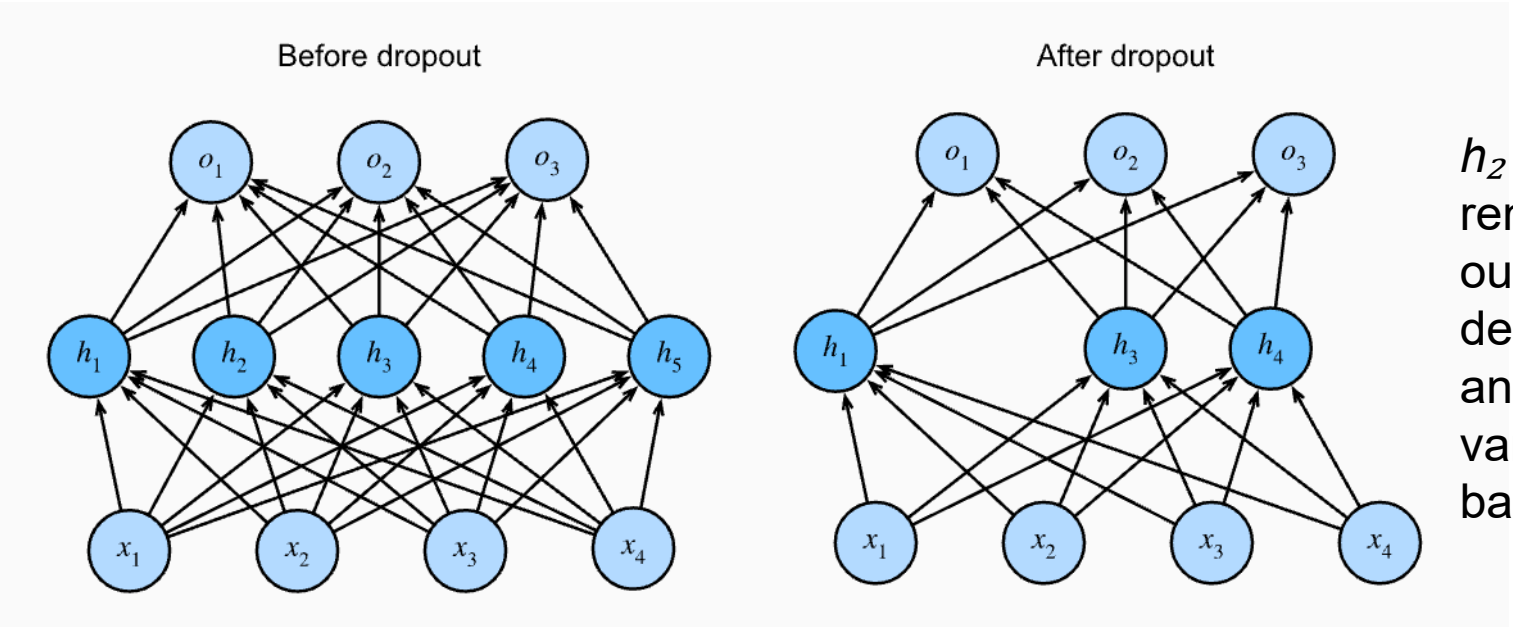
Ridge regularization prevents overfitting

Dropout

Dropout: One randomly sets a fraction of the nodes or neurons (p) in each layer to zero and compensates for the reduced capacity by scaling the remaining activations inversely to the retention fraction (h').

$$h' = \begin{cases} 0 & \text{with probability } p. \\ h/(1-p) & \text{otherwise} \end{cases}$$

- The expectation values of h and h' are the same.



h_2 and h_5 are removed, so the outputs no longer depend on them, and their gradients vanish during backpropagation.

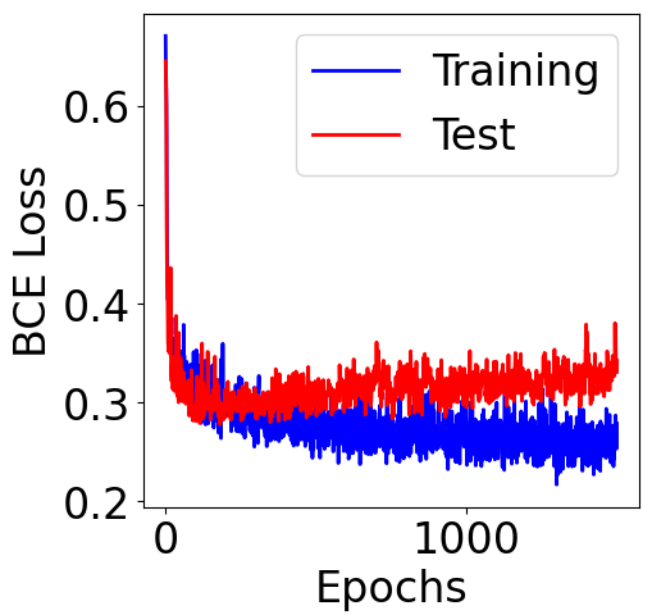
Dropout

Dropout: One randomly sets a fraction of the nodes or neurons (p) in each layer to zero and compensates for the reduced capacity by scaling the remaining activations inversely to the retention fraction (h').

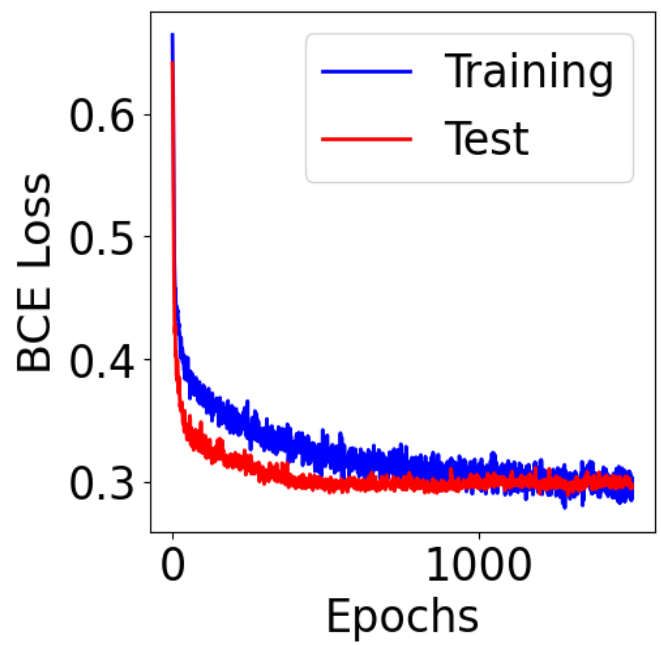
$$h' = \begin{cases} 0 & \text{with probability } p. \\ h/(1-p) & \text{otherwise} \end{cases}$$

- The expectation values of h and h' are the same.

No regulation



Dropout probability: p=50%



Dropout

Dropout: One randomly sets a fraction of the nodes or neurons (p) in each layer to zero and compensates for the reduced capacity by scaling the remaining activations inversely to the retention fraction (h').

$$h' = \begin{cases} 0 & \text{with probability } p. \\ h/(1-p) & \text{otherwise} \end{cases}$$

- The expectation values of h and h' are the same.

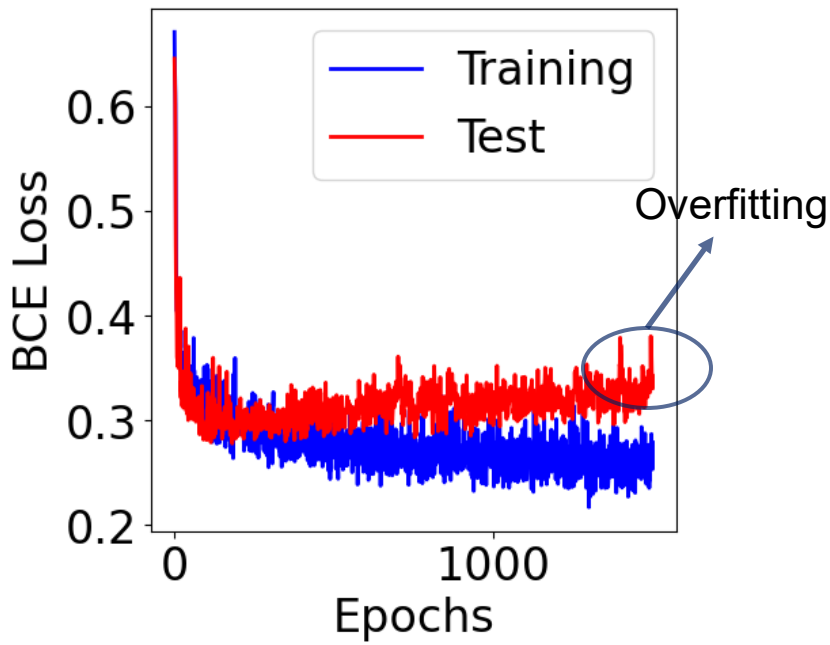
- Prevents neurons from co-adapting too much, encouraging the network to learn more robust features.

- Particularly helpful when training deep neural networks or when dealing with noisy chemical data, improving the prediction.

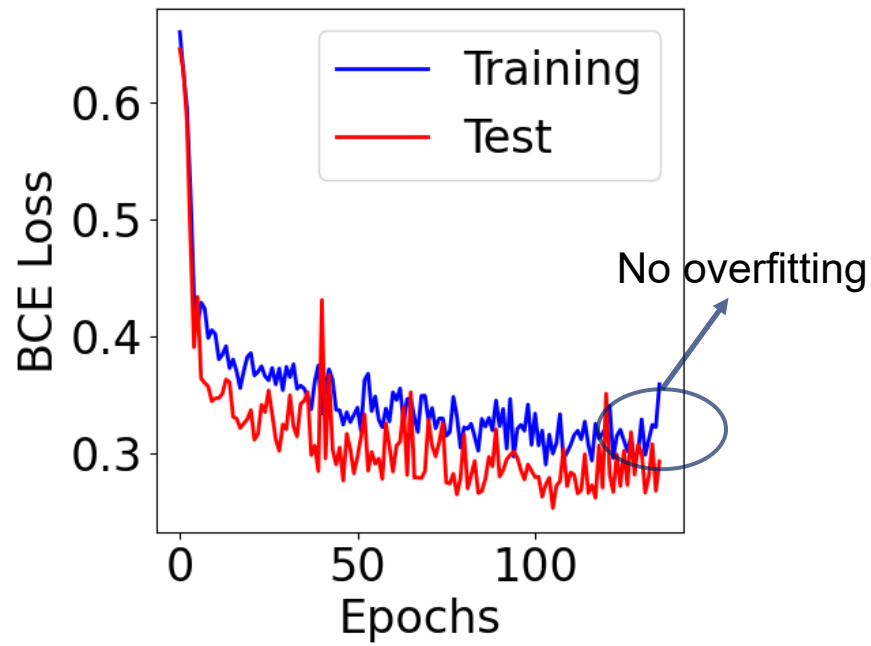
Early Stopping

Early stopping: monitor the validation loss during training and stopping when it starts to increase, indicating overfitting.

No regulation



Early Stopping if validation loss does not increase for 30 epochs



Save computational resources by preventing excessive training that leads to overfitting.

Summary: Neural Network

Pros:

- Effective at approximating complex **nonlinear functions**.
- **Straightforward parameter optimization** using backpropagation and the chain rule.
- **Highly flexible** for various data types (e.g., molecular structures, reaction outcomes).

Cons:

- Stacking many hidden layers can lead to **vanishing/exploding gradients**.
- Training may get stuck in local minima.
- A large number of parameters can cause **overfitting**.
- Require **large datasets** and computational resources.
- **Lack of interpretability** can hinder understanding of model decisions.